



信息与软件工程学院

挑战性项目 II 总结报告

课程名称： 进阶式挑战性综合项目 II（嵌入式系统）

所在系别： 软件工程（嵌入式系统）

学 期： 2024-2025 学年 2 学期

课题名称： 四轴飞行器设计

指导教师： 廖勇

学生信息：

序号	学号	姓名
1（组长）	2023090903028	曾欣晨
2	2022130102013	成棋伟
3	2023090903010	吴欣锐
4	2023090901015	冉昕堃

电子科技大学信息与软件工程学院

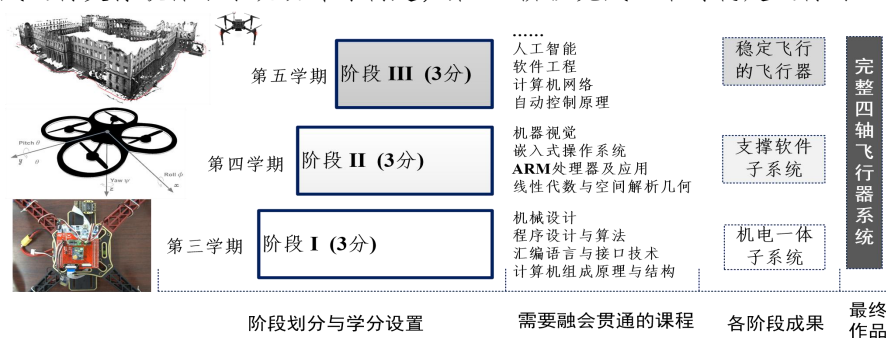
进阶式挑战性综合项目 II 课题任务书

课题名称	四轴飞行器设计				
课程名称	进阶式挑战性综合项目 II	专业方向	嵌入式系统	选课年级	2023 级
指导教师	廖勇	教师电话	13350850928	教师邮箱	Liaoyong@uestc.edu.cn

主要任务（请注意内容与工作量要求并覆盖毕业要求相关指标点，参见背页说明）：

1 四轴飞行器设计总体任务

基于 STM32 嵌入式开发平台，以 3-6 个同学为单位，设计并实现一个四轴飞行器。综合课程设计跨度 1.5 年，分为三个阶段实施：I、II、III，分别在第 3、第 4、第 5 学期完成。第 I 阶段完成机电一体化子系统设计；第 II 阶段完成运行支撑软件子系统设计与构建；第 III 阶段完成整个可稳定飞行的四轴飞行器设计。



2 四轴飞行器设计第 II 阶段任务（进阶式挑战性综合项目 II）

在前阶段 I（进阶式挑战性综合项目 I）基础上，掌握嵌入式实时操作系统启动、加载、移植、多任务并发执行，嵌入式实时操作系统驱动程序设计，基于嵌入式实时操作系统多任务程序设计等重要内容，并通过项目实战实现，在自己完成的硬件系统上移植嵌入式实时操作系统，再对姿态传感器数据获取与处理、蓝牙、接收机、电机等驱动在有嵌入式操作系统环境下进行重构。设计并实现任务上下文切换、嵌入式实时操作系统时间触发机制、事件触发机制、系统启动与加载的关键任务。

预期成果或目标：

本学期在综合课程设计 I 硬件子系统基础上，持续完成嵌入式实时操作系统移植及基础软件重构，与前一学期成果进行整合，实现四轴飞行器运行支持子系统构建。

涉及知识点：

嵌入式实时操作系统：多任务并发执行、操作系统移植、时间与事件触发机制、BOOTLOADER 等
嵌入式驱动程序设计：加速度计、陀螺仪、地磁仪、蓝牙、电机、接收机、超声波传感器等
编译技术：编译器、连接器、嵌入式交叉开发工具链（Linux 嵌入式开发工具链、System View）
计算机系统结构、电子电路基础：STM32 处理器结构，I/O 设备、存储机制等

指导教师签名：_____

年 月 日

备注：此任务书必须双面打印。

毕业要求指标点映射图（进阶式挑战性综合项目 II）

进阶式挑战性综合项目 II 面向中年级学生开设，要求学生在学习相关课程后参与一个中等难度的小型软件工程项目，工作重点在于学生利用软件工程的思想进行系统设计与系统实现，并体现一定的创新意识（要求同学完成软件工程项目的阶段，但考核重点放在系统设计与系统实现阶段）。

工作内容与工作量要求	对应指标点
1、总体设计（概要设计）阶段能够复杂软件工程问题进行模块分解，并且设计出满足特定需求的总体设计方案；	GR3.3 学生能够针对复杂软件工程问题，设计满足特定需求的总体设计和详细设计 GR3.4 学生能够集成单元过程进行软件系统流程设计，对流程设计方案进行优选，体现创新意识 GR5.2 能够根据软件系统的应用场景，选择合适的开发环境、工具与技术标准进行软件系统的开发 GR9.2 学生能够独立完成团队分配的工作，并能胜任团队成员角色，承担相应责任 GR10.2 学生能够进行陈述发言，清楚表达对复杂软件工程问题的看法与见解
2、详细设计阶段能够针对复杂软件工程问题设计出满足特定需求的详细设计方案；详细设计阶段能够集成单元过程对软件系统的流程进行设计，并且选出一种最优的流程设计方案，能够体现创新意识；	
3、编码阶段能够根据软件系统的应用场景，选择合适的开发环境、工具与技术标准进行软件系统的开发；	
4、综合项目报告能够体现出综合项目课题小组团队分工以及每位组员独立完成的工作；	
5、答辩阶段，能够进行陈述发言，清楚表达对复杂软件工程问题的看法与见解。	

电子科技大学信息与软件工程学院

进阶式挑战性综合项目 II 中期考核表

课题名称	四轴飞行器设计				
所在系别	软件工程 (嵌入式系统)	指导教师	廖勇	执行学期	2024-2025 学年 2 学期
序号	学号			姓名	
1 (组长)	2023090903028			曾欣晨	
2	2022130102013			成棋伟	
3	2023090903010			吴欣锐	
4	2023090901015			冉昕堃	
5					
6					
序号	评价项目	分项占比	评判标准		得分
1	针对工程问题的方案设计	0.2	能够针对复杂软件工程问题进行推理分析,设计满足需求的总体设计和详细设计		优秀[18,20] 良好[14,18] 中等[10,14] 较差[06,10] 不及格[0,6)
2	针对工程问题的推理分析	0.2	能够集成单元过程进行软件系统流程设计,对流程设计方案进行优选		优秀[18,20] 良好[14,18] 中等[10,14] 较差[06,10] 不及格[0,6)
3	针对工程问题的具体实现	0.2	能够根据软件系统的应用场景,选择合适的开发环境、工具与技术标准进行软件系统的开发		优秀[18,20] 良好[14,18] 中等[10,14] 较差[06,10] 不及格[0,6)
4	知识技能学习情况	0.1	能对文献和书籍进行查阅、分析和总结,寻求相应问题的解决方案		优秀[9,10] 良好[7,9] 中等[5,7] 较差[3,5] 不及格[0,3)

电子科技大学信息与软件工程学院
进阶式挑战性综合项目 II 成绩考核表

课题名称	四轴飞行器设计					
所在系别	软件工程 (嵌入式系统)	指导教师	廖勇	执行学期	2024-2025 学年 2 学期	
成绩汇总表						
学号	姓名	中期考核 (10 分)	总结报告 (40 分)	组内贡献(10 分/人)	答辩成绩 (40 分)	总分 (100 分)
2023090903028	曾欣晨					
2022130102013	成棋伟					
2023090903010	吴欣锐					
2023090901015	冉昕堃					
指导教师签名		年 月 日				
“成绩汇总表”填表说明：此表中的学号和姓名由学生填写，后面的分项和总分由指导教师填写；中期考核成绩、总结报告成绩、综合项目答辩成绩请填写按百分制得分折算后的分数。						
(一)组内贡献互评						
学号	姓名	承担的主要工作				个人贡献 (10 分/人)
2023090903028	曾欣晨	操作系统移植和调试，系统工程重构				10
2022130102013	成棋伟	操作系统移植和调试，启动代码编写				10
2023090903010	吴欣锐	上下文切换与临界区，演示报告编纂				9
2023090901015	冉昕堃	事件驱动与时钟驱动，总体报告撰写				9
学生签名		年 月 日				
指导教师签名		年 月 日				
评判标准		贡献非常突出[9,10]，突出[7,8]，一般[5,6]，较少[3,4]，非常少[0,2]				

(二)综合项目报告					
序号	评价项目	分项占比	评判标准		得分
1	针对复杂工程问题的方案设计	0.3	能够针对复杂软件工程问题,设计满足特定需求的总体设计和详细设计,设计方案合理可实现	优秀[25,30] 良好[20,25] 中等[15,20] 较差[8,15] 不及格[0,8)	
2	针对复杂工程问题的推理分析	0.2	能够集成单元过程进行软件系统流程设计,对流程设计方案进行优选。优选方案合理,且具有一定创新意识	优秀[18,20] 良好[14,18) 中等[10,14) 较差[06,10) 不及格[0,6)	
3	针对复杂工程问题的方案实现	0.3	能够根据软件系统的应用场景,选择合适的开发环境、工具与技术标准进行软件系统的开发;代码规范、功能完整,能够达到设计目标	优秀[25,30] 良好[20,25) 中等[15,20) 较差[8,15] 不及格[0,8)	
4	知识技能学习情况	0.1	能对文献和书籍进行查阅、分析和总结,寻求相应问题的解决方案	优秀[9,10] 良好[7,9) 中等[5,7) 较差[3,5) 不及格[0,3)	
5	工程文档写作与工程协作交流	0.1	报告书结构严谨,逻辑性强,论述层次清晰,语言准确,文字流畅,符合规范要求;术语、图表等符合标准;文档能够体现团队协作交流情况;能够体现个人独立完成团队分配的工作,并能胜任团队成员角色,承担相应责任	优秀[9,10] 良好[7,9) 中等[5,7) 较差[3,5) 不及格[0,3]	
总分(满分100分)					
指导教师评语		进阶式挑战性综合项目II完成情况、报告情况、分工合作情况等综合评价			
		指导教师签名:			
		年 月 日			

(三) 综合项目答辩				
序号	评价项目	分项占比	评判标准	得分
1	进阶式挑战性综合项目 II 完成情况陈述	0.4	报告过程中思路清晰、条理清楚，语言流畅，能清楚表达对复杂软件工程问题的看法与见解；能在规定的时间内能流畅正确地报告综合项目的主要工作及成绩	优秀[33,40] 良好[25,33) 中等[17,25) 较差[9,17) 不及格[0,9)
2	现场演示	0.3	课题需求的基本功能是否实现，演示是否充分；是否在基本功能之外进行了优化或实现额外功能	优秀[25,30] 良好[20,25) 中等[15,20) 较差[8,15) 不及格[0,8)
3	回答问题	0.15	能够回答与实习内容相关的基本性和扩展性问题；了解软件工程领域国际发展前沿状况，能够就涉及的复杂软件工程问题表达自己的想法；回答问题时思维活跃、反映敏捷、表述有条理，无明显错误	优秀[13,15] 良好[10,13) 中等[7,10) 及格[4,7) 不及格[0,4)
4	答辩材料	0.15	答辩材料内容充实，图文并茂，有足够的难度和工作量，演示效果好，论述及结论正确	优秀[13,15] 良好[10,13) 中等[7,10) 及格[4,7) 不及格[0,4)
总分（满分 100 分）				
进阶式挑战性综合项目 II 答辩组评语		1. 进阶式挑战性综合项目 II 报告的逻辑思维、工作量及任务完成等情况 2. 学生回答问题时所反映的逻辑思维、基本知识、基本技能和知识面等情况 答辩组教师签名： 年 月 日		

备注：1. 成绩考核表所有签名处需由相关人员亲笔签名。

2. 成绩考核表需双面打印。



信息与软件工程学院

挑战性项目 II 总结报告

课程名称： 进阶式挑战性综合项目 II（嵌入式系统）

所在系别： 软件工程（嵌入式系统）

学 期： 2024-2025 学年 2 学期

课题名称： 四轴飞行器设计

指导教师： 廖勇

学生信息：

序号	学号	姓名
1（组长）	2023090903028	曾欣晨
2	2022130102013	成棋伟
3	2023090903010	吴欣锐
4	2023090901015	冉昕堃

摘 要

本报告主要探讨了在基于综合课程设计 I 内容的前提下，将成熟的 uC/OS II 嵌入式操作系统移植到四轴飞行器硬件平台上，并合理划分业务层任务，使得在操作系统管理下对各任务进行合理调度，提升 MCU 处理效率，增强实时性。

为完成这个课题，我们深入学习 Cortex-M4 内核，结合 ARM 处理器汇编知识，分析微处理器的启动过程，了解堆、栈、中断向量表等概念并进行实践。同时，我们针对操作系统的事件驱动机制和时间驱动机制进行了研究，实践了裸板汇编中断实验，并对中断响应的全过程进行了全方位的了解。

在以上基础上，我们通过自主实现上下文切换代码、进出临界区代码、堆栈初始化代码、时钟驱动代码等，让 uC/OS II 操作系统能够在 STM32F4 处理器上正常运行并执行业务。通过在代码中插入 SystemView 代码跟踪工具，能够连接上位机以直观的图形化方式观察操作系统运行情况，帮助分析。

关键词：四轴飞行器、STM32、Cortex-M4 内核，uC/OS-II

目 录

摘 要	I
目 录	II
第一章 针对复杂工程问题的方案设计和实现	- 1 -
1.1 针对复杂工程问题的方案设计	- 1 -
1.1.1 阶段性目标分析	- 1 -
1.1.2 操作系统和调式模块的选择	- 1 -
1.1.3 总体设计	- 2 -
1.2 针对复杂工程问题的详细分析	- 4 -
1.2.1 $\mu\text{C}/\text{OS II}$ 操作系统架构	- 4 -
1.2.2 OS 多任务调度设计	- 5 -
1.2.3 ARM 内核事件驱动机制	- 9 -
1.2.4 ARM 内核时间驱动机制	- 12 -
1.2.5 $\mu\text{C}/\text{OS II}$ 启动代码设计	- 15 -
1.2.6 $\mu\text{C}/\text{OS II}$ 内核移植设计	- 19 -
1.2.7 SystemView 模块移植设计	- 20 -
1.3 针对复杂工程问题的方案实现	- 21 -
1.3.1 用 KEIL 重构 $\mu\text{C}/\text{OS II}$ 新工程	- 21 -
1.3.2 事件驱动实现	- 26 -
1.3.3 时间驱动实现	- 31 -
1.3.4 $\mu\text{C}/\text{OS II}$ 在 STM-32 上的移植实现	- 35 -
1.3.5 SystemView 模块移植和使用	- 39 -
第二章 系统测试	- 44 -
2.1 $\mu\text{C}/\text{OS II}$ 移植测试	- 44 -
2.1.1 测试目的	- 44 -
2.1.2 测试步骤	- 44 -
2.1.3 测试代码	- 44 -
2.1.4 预期结果	- 45 -
2.1.5 测试结果	- 45 -
2.2 项目集成测试	- 45 -
2.2.1 测试目的	- 45 -
2.2.2 测试步骤	- 45 -
2.2.3 预期结果	- 46 -
2.2.4 测试结果	- 46 -
第三章 知识技能学习情况	- 48 -
3.1 互斥与临界区	- 48 -

3.1.1 互斥与临界区概念	- 48 -
3.1.2 三种进入临界区的方式	- 48 -
3.2 MAKEFILE 与 CMAKE 工程构建	- 50 -
3.2.1 程序编译过程	- 50 -
3.2.2 交叉编译工具链	- 50 -
3.2.3 Makefile 基础	- 51 -
3.2.4 Cmake 工程构建	- 52 -
参考文献	- 54 -
致谢	- 55 -

第一章 针对复杂工程问题的方案设计和实现

1.1 针对复杂工程问题的方案设计

1.1.1 阶段性目标分析

此次项目，我们小组的设计思路分为：硬件模块功能，操作系统移植统筹，电机控制算法。在上学期的项目中，我们已实现了硬件等响应模块的功能，在开启本学期的任务之前，我们先回顾一下第一部分的成果：

- 四轴飞行器机架搭建
- PCB 转接板的设计，出板与焊接
- GY-86 的数据获取与分析
- 捕获遥控器接收机产生的 PWM 波信号
- 通过 STM32 对应引脚输出 PWM 波信号调节舵机转速

在上一学期的设计中，我们使用了循环来完成类似于操作系统的轮询操作，但这种方式会导致 CPU 因长时间等待信号被占用，并不能满足本挑战项目最后所要达到的实时性能，也不方便我们下学期添加新的姿态解算和 PID 反馈控制等任务。

为了使飞行器系统能够根据任务的重要性的时间紧迫性有逻辑，有次序的稳定运行，我们需要将整体架构重构，完成实时操作系统的移植。于是在本进阶式挑战性综合项目 II 的课程中，基于上一学期挑战性课程的裸机基础之上，我们需要实现实时操作系统的移植。

为了便于调试，我们要引入任务级调试工具以观察每个任务的运行情况，找出问题并进行优化改进。

1.1.2 操作系统和调式模块的选择

操作系统是管理计算机硬件和软件资源的应用程序，它需要分配 CPU 的资源、统筹资源供需的先后顺序、控制输入输出设备、管理文件系统等基本事务。在四轴飞行器上，用户希望在同一时间内有多个任务并发执行，而 MCU 一次只能顺

序执行其中的一项任务。为了使各个程序能运行且正确地运行，需要合理的分配先后顺序，从而让每个任务都得到足够的资源去运行，保证飞行器的稳定运行。

考虑到 STM32 本身的内存限制，以及对于实时性性能的要求，我们选择了 $\mu\text{C}/\text{OS II}$ 这一个操作系统。它包含了任务调度、任务管理、时间管理、内存管理、任务间通信等多种功能，而且稳定性强，代码开源，生态良好、资料丰富，方便初学者上手移植实现。

选定操作系统后，我们还需要查看各个任务在这个操作系统上的运行情况，例如运行开始时间，运行时间，抢占情况等。本小组采用的是名为 Systemview 的操作系统监视分析工具，它能够以条形图的形式，将所有任务的运行时间呈现出来，且与当前系统兼容良好，极大提高了开发效率。

1.1.3 总体设计

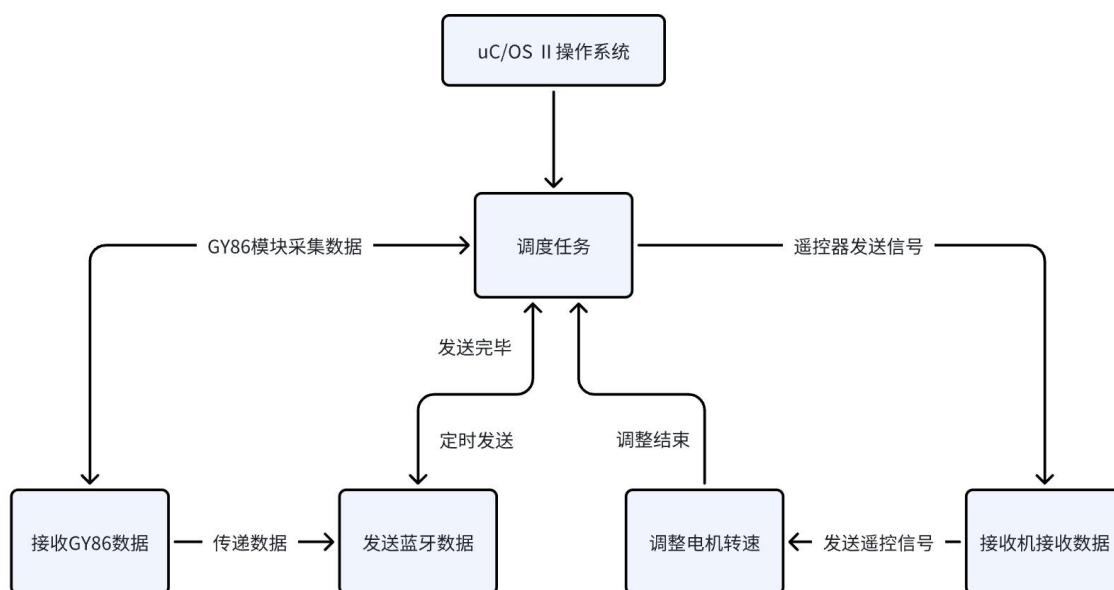


图 1-1 总体任务架构图

本次课程项目的总体设计如图 1-1 所示。为了使各个硬件模块之间能够形成良好的逻辑顺序，我们将四个硬件模块分为初始化和运行任务两个不同的函数。初始化用于在操作系统启动时就配置好，运行代码则是要等开始运行操作系统后才创建任务。所以我们就有了如上的架构，将以往四个模块，分配成操作系统下的两个任务，一个任务负责改变电机转速，另一个任务负责读取 GY86 数据并通过蓝牙发送至上位机，这样的架构不仅方便从系统中移除，也方便以后有新的任务（如姿态解算）添加时，可以更加简单高效，不易出错。

为了实现 $\mu\text{C}/\text{OS}$ 下的多任务并发集成，我们综合考虑硬件模块初始化、数据传输需要以及电机控制逻辑，决定设计的系统业务逻辑如下：

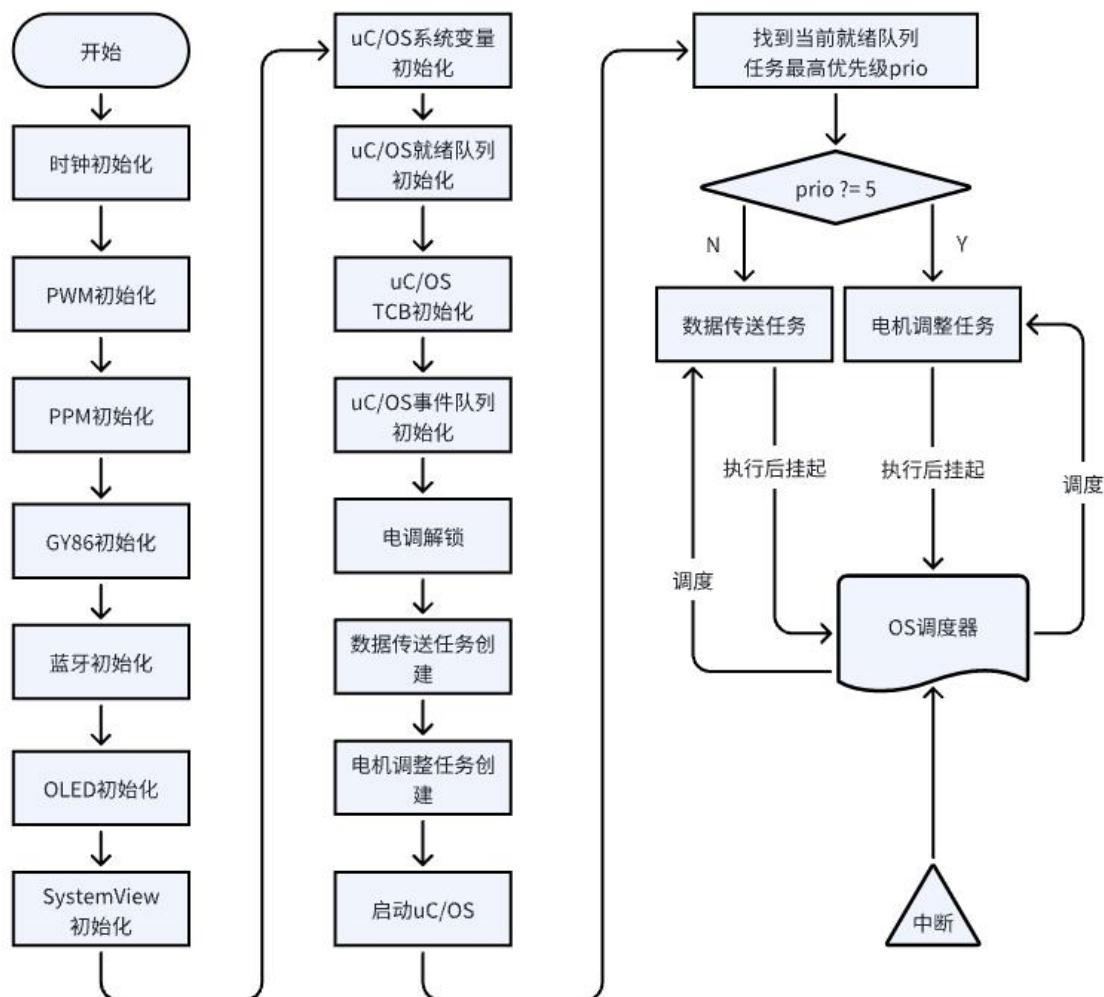


图 1-2 业务层逻辑流程图

如图所示，我们的业务逻辑首先是依次完成时钟、PWM、PPM、GY86、蓝牙、OLED 及 SystemView 的初始化，随后完成 $\mu\text{C}/\text{OS}$ 系统变量、就绪队列、TCB、事件队列的初始化，电调解锁，创建任务并启动 $\mu\text{C}/\text{OS}$ 。在完成配置后，系统会判断当前就绪队列任务最高优先级，并进行任务调度；任务部分我们主要设计并实现了两大任务，分别为数据传送任务和电机调整，其优先级分别为 5 和 6，并在完成执行后挂起，等待下一次被调度。OS 调度器同时随时接受中断请求并完成中断响应。

1.2 针对复杂工程问题的详细分析

1.2.1 $\mu\text{C}/\text{OS II}$ 操作系统架构

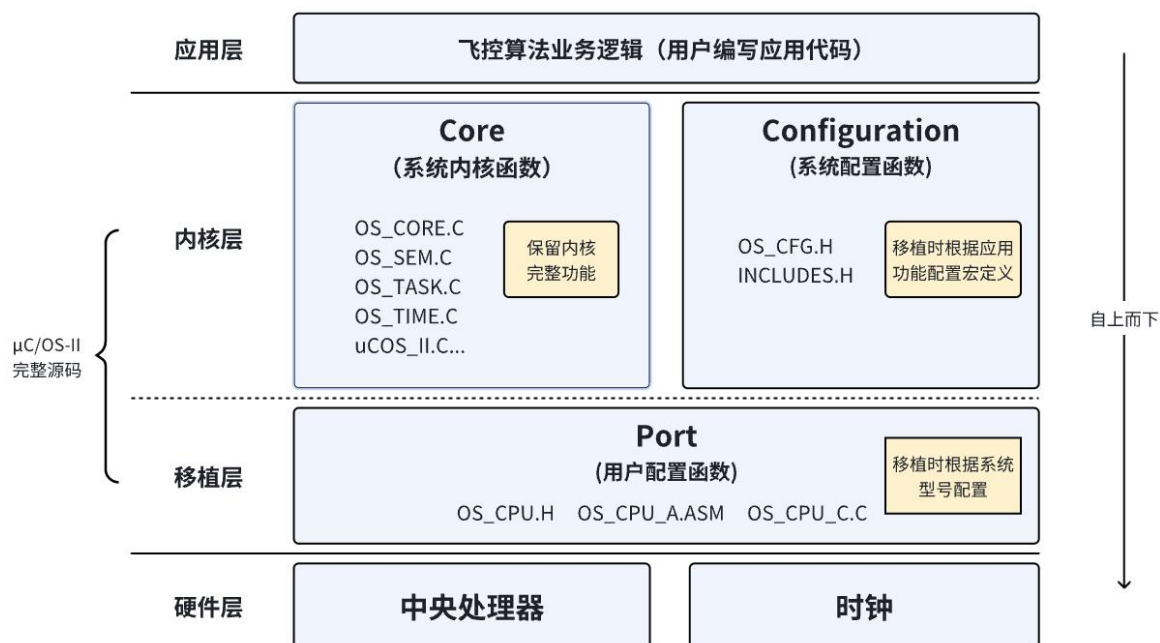


图 1-3 $\mu\text{C}/\text{OS II}$ 内核架构图

在利用 $\mu\text{C}/\text{OS II}$ 进行设计时，我们首先需要学习 $\mu\text{C}/\text{OS II}$ 内核结构、功能和函数，并且能够针对 STM32F401 开发板进行合理的设置与移植。以下是对 $\mu\text{C}/\text{OS II}$ 源代码内核的各个功能模块的详细介绍。

1) CORE（核心，一般不作修改）

① OS_CORE.c: $\mu\text{C}/\text{OS II}$ 内核的核心初始化文件，负责初始化任务调度器、创建 Idle 任务、事件控制块等。

② OS_TASK.c: 包含任务管理相关的函数，如任务的创建、删除、修改任务控制块（TCB）内容、任务延时等相关功能。

③ OS_FLAG.c: 用于实现标志（Flag）机制，服务于任务间的通信和同步。标志可以被设置、等待和清除。

④ OS_MEM.c: 实现与内存管理相关的函数，可以动态分配和释放内存块，并提供了内存池和固定大小内存块分配策略。

⑤ OS_TMR.c: 实现定时器功能，允许用户设置定时器，以在指定的时间点或

时间间隔后触发相应的操作或任务。

⑥ OS_MUTEX.c: 用于实现互斥量 (Mutex) 机制, 提供实现互斥量的接口函数, 服务于任务间互斥访问共享资源。

⑦ OS_SEM.c: 用于实现信号量 (Semaphore) 机制, 提供实现信号量的接口函数, 服务于任务间的同步和互斥。

⑧ OS_MBOX.c: 用于实现邮箱 (Mailbox) 机制, 提供实现邮箱功能的接口函数, 服务于任务间的消息传递。

⑨ OS_Q.c: 用于实现队列 (Queue) 机制, 提供实现队列的接口函数, 服务于任务间通信。

⑩ uCOS_II.h: 包含 μ C/OS II 内核中各种数据结构的定义和函数声明, 如任务、事件、链表、信号量等。

2) PORT (根据处理器不同需要调整实现正确移植)

① OS_CPU_C.c: 包含与特定处理器相关的代码, 提供钩子函数、任务栈的初始化和系统时钟 Tick 的配置等。

② OS_CPU_A.asm: μ C/OS II 中汇编实现的上下文切换核心代码, 负责保存和恢复任务的上下文, 并通过调用 C 语言实现的函数完成任务切换操作。

3) CONFIG

① includes.h: 包含所有 μ C/OS II 的头文件和与处理器相关的头文件。

② os_cfg.h: μ C/OS II 提供了裁剪功能, 可以根据系统需求选择性地裁剪内核功能。这个文件用于配置这些功能的开启和关闭。

1.2.2 OS 多任务调度设计

1) Cortex-M4 寄存器组织

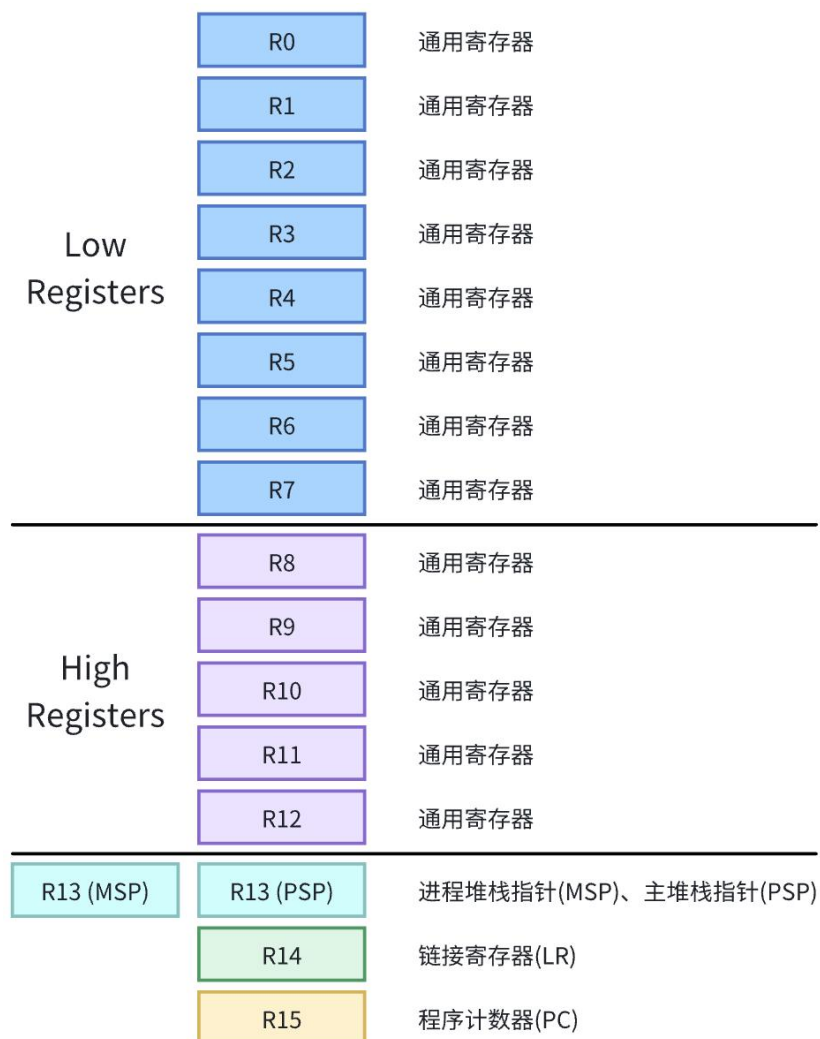


图 1-4 Cortex-M4 寄存器组织图

Cortex-M4 有类似于 ARM9 的寄存器，它拥有 R0-R15 共 16 个寄存器。其中，R13 被用作堆栈指针，用于指示当前任务或代码的堆栈位置，由于 Cortex-M4 处理器内核支持多种工作模式，因此存在两个堆栈指针 MSP (Main Stack Pointer) 和 PSP (Process Stack Pointer)；R14 为链接寄存器 LR (Link Register)；R15 为程序计数器 PC (Program Counter)。

MSP 是主堆栈指针，用于 Handler 模式下的堆栈操作。这个模式一般由操作系统内核、异常服务程序以及需要特权访问的应用程序代码使用。当发生异常或中断时，处理器会自动切换 Handler 模式，并使用 MSP 进行堆栈操作。Handler 模式下的堆栈用于保存当前上下文信息，包括寄存器值、函数调用信息等。

PSP 是线程模式下的堆栈指针，用于普通线程模式下任务的堆栈操作。在线程

模式下，每个任务都有自己的堆栈空间，用于保存该任务的上下文信息。PSP 可以被任务切换操作使用，以保存和恢复任务的堆栈状态。相比于 Handler 模式下的主堆栈指针 MSP，PSP 更加轻量级，适用于简单的应用场景。

设置两个堆栈可能的原因为：为了在进行模式转换的时候，减少堆栈的保存工作；同时也可以为不同权限的工作模式设置不同的堆栈。

2) PendSV 异常

PendSV 是 Cortex-M3/4 处理器提供的一种特殊系统调用，它可以被用来延迟执行一个异常，直到其他紧急任务完成后再执行操作。在任务上下文切换的汇编程序中，我们可见 OSCtxSw 的代码只有短短几行，因为它只是触发了一个 PendSV 中断，具体的切换过程其实是在 PendSV 服务函数里面进行的。

要触发 PendSV，只需将中断控制及状态寄存器 ICSR 的第 28 位 PENDSVSET 设置为 1 即可。当该位被设置后，处理器会在合适的时机引发 PendSV 异常（通常是当前指令执行完毕或者发生中断返回时）。

通过利用 PendSV 异常，操作系统可以灵活地实现任务调度和上下文切换，以满足不同任务的优先级和时间需求。同时，PendSV 异常的使用也减少了对处理器硬件和指令的依赖，使任务切换过程简洁高效。

位	名称	类型	复位值	描述
31	NMIPENDSET	R/W	0	置 1 挂起 NMI
28	PENDSVSET	R/W	0	置 1 挂起系统调用
27	PENDSVCLR	W	0	置 1 清除 PendSV 挂起状态
26	PENDSTSET	R/W	0	置 1 挂起 PendSV 异常
25	PENDSTCLR	W	0	置 1 清除 SYSTICK 挂起状态
23	ISRPREEMPT	R	0	挂起中断下一步变为活跃状态
22	ISRPENDING	R	0	外部中断挂起
21:12	VECTPENDING	R	0	挂起的 ISR 编号

表 1-1 中断控制及状态寄存器 ICSR

3) 中断状态寄存器及其相关定义

在 `os_cpu_a.asm` 中，有以下四个寄存器的定义：

代码 1-1 中断状态相关定义

<code>SCB_PRI_REG_PENDSV</code>	<code>EQU</code>	<code>0xE000ED22</code>	; interrupt priority register
<code>NVIC_ICS_REG</code>	<code>EQU</code>	<code>0xE000ED04</code>	; interrupt control state register
<code>NVIC_PENDSV_PRI</code>	<code>EQU</code>	<code>0x000000FF</code>	; PendSV priority
<code>NVIC_PENDSV_SETPEND</code>	<code>EQU</code>	<code>0x10000000</code>	; PendSV set to pend status

这些定义的作用是用于中断管理和控制。`SCB_PRI_REG_PENDSV` 用于配置 PendSV 中断的优先级，地址为 `0xE000ED22`；`NVIC_ICS_REG` 是中断控制状态寄存器（Interrupt Control State Register）的地址，用于管理和控制中断的状态；`NVIC_PENDSV_PRI` 用于设置 PendSV 中断的优先级，其定义值为 `0x000000FF`，即最高优先级；`NVIC_PENDSV_SETPEND` 用于触发 PendSV 中断。

4) 任务调度

$\mu\text{C}/\text{OS-II}$ 是一个基于任务优先级抢占式任务调度的实时操作系统。它采用优先级抢占式调度策略，即在内核在进行调度管理时，调用任务切换函数 `OSSched()`，确保此时处于就绪状态且为最高优先级的任务能够立即得到 CPU 执行，并通过任务控制块（Task Control Block, TCB）来管理和调度任务。

在 $\mu\text{C}/\text{OS-II}$ 中存在多个任务调度点，总结后可归类为以下三种情况：

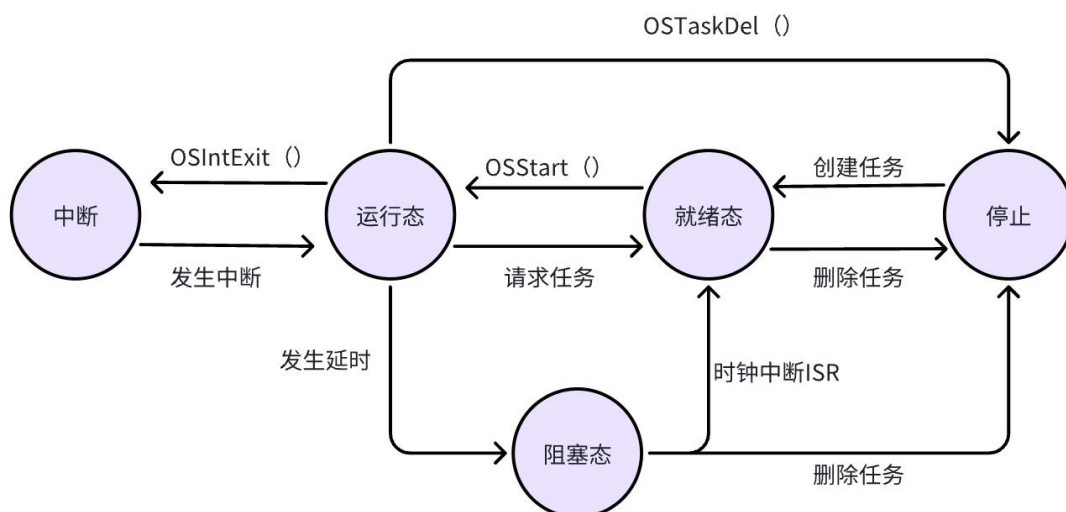


图 1-5 $\mu\text{C}/\text{OS-II}$ 任务调度点

① **时钟节拍中断**: $\mu\text{C}/\text{OS-II}$ 周期性调用时钟节拍中断服务函数 `OSTickISR()`。该中断服务函数负责更新任务的延时计数器 `OSTimeTick()`。当任务的延时时间到达时, `OSTickISR()` 随即调用任务切换函数 `OSSched()` 进行任务切换, 避免单个任务长时间占用 MPU 资源。

② **时间延迟函数**: 在任务中, 可以调用时间延迟函数 `OSTimeDly()` 来挂起当前任务并设置延时时间。当调用 `OSTimeDly()` 时, 当前任务被挂起, 其任务控制块中的相应信息被保存, 然后调用任务切换函数 `OSSched()` 进行任务切换。`OSSched()` 会选择就绪任务表中优先级最高的任务进行执行。

③ **退出中断**: 当嵌套中断退出时, $\mu\text{C}/\text{OS-II}$ 会在 `OSIntExit()` 函数中执行任务切换。在退出中断时, 如果有更高优先级的任务处于就绪状态, 则执行任务切换。

在任务调度过程中, $\mu\text{C}/\text{OS-II}$ 需要找到当前最高优先级的就绪任务, 然后将 CPU 的控制权交给该任务进行执行。具体来说, 系统根据任务就绪表找到最高优先级任务标识 (即它的优先级), 进而获得该任务的任务控制块。 $\mu\text{C}/\text{OS-II}$ 通过优先级位图法记录 64 个不同的优先级(0-63), 在任务调度时其通过以下两行代码找到最高优先级任务。

代码 1-2 中断状态相关定义

```
y = OSUnMapTbl[OSRdyGrp];
OSPrioHighRdy = (INT8U)((y<<3)+ OSUnMapTblI[OSRdyTbl[y]]);
```

其中: 优先级判定表 (`OSUnMapTbl`) 是 $\mu\text{C}/\text{OS-II}$ 中的一个查找表, 用于将任务组的位映射为对应的优先级值。在获得了最高就绪任务的任务控制块指针后, 加上存放在指针变量 `OSTCBCur` 中的当前运行任务的任务控制块, 就可以进行任务切换的工作了。

1.2.3 ARM 内核事件驱动机制

在 ARM 处理器中, 事件驱动是指将可能发生的事情进行编号, 通过这些编号, 我们可以设置硬件的跳转地址。当这些预设的事件发生时, 由硬件发出信号, 告知 MCU 发生了什么, 再通过指令跳转到这事件预设的中断服务函数中完成对应的处理。

Cortex-M4 内核中跟中断相关的部分主要是 NVIC(嵌套向量中断控制器)管理模块, 它负责控制来自内核内部的异常和来自外部的中断, 最多能够支持 240 个 IRQ(中断请求)、1 个不可屏蔽中断(NMI)、1 个 SysTick(系统节拍)定时中断及多个

系统异常。多数 IRQ 由定时器、I/O端口和通信接口(如 UART 和 IC)等外设产生。NMI 通常由看门狗定时器或掉电检测器等外设产生，其余的异常则是来自处理器内核，中断还可以利用软件生成。

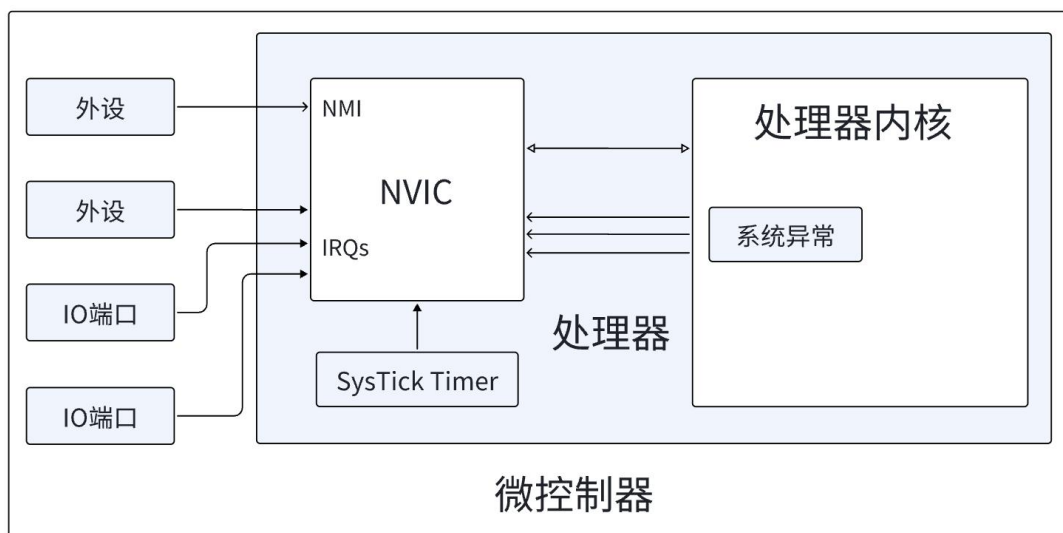


图 1-6 典型 MPU 中的各种异常源

1) NVIC 相关内容

NVIC 在软件的角度上看，是一个被映射成外设的处理器内部模块，编程人员可以像访问外设一样去操控配置 NVIC 模块。其具备灵活的中断和异常管理功能，且通过可编程优先级支持中断嵌套（抢占）。

表 1-2 用于中断控制的 NVIC 寄存器列表

地址	寄存器	CMSIS-Core 符号	功能
0xE000E100~ 0xE000E11C	中断设置使能寄存器	NVIC->ISER[0] ~ NVIC->ISER[7]	写 1 设置中断使能位
0xE000E180~ 0xE000E19C	中断清除使能寄存器	NVIC->ICER[0] ~ NVIC->ICER[7]	写 1 清除中断使能位
0xE000E200~ 0xE000E21C	中断设置挂起寄存器	NVIC->ISPR[0] ~ NVIC->ISPR[7]	写 1 设置中断挂起状态位
0xE000E280~ 0xE000E29C	中断清除挂起寄存器	NVIC->ICPR[0] ~ NVIC->ICPR[7]	写 1 清除中断挂起状态位

0xE000E300~ 0xE000E31C	中断活跃位寄存器	NVIC->IABR[0] ~ NVIC->IABR[7]	指示中断当前活跃状态,只 读
0xE000E400~ 0xE000E4EF	中断优先级寄存器	NVIC->IP[0] ~ NVIC->IP[239]	配置每个中断的优先级 (8 位宽)
0xE000EF00	软件触发中断寄存器	NVIC->STIR	写入中断编号(0-239)以设 置相应中断的挂起状态

2) EXTI 外设

EXIT 外设为 Cortex-M4 内核外的一个功能模块,负责响应所有外部中断信号。在下图中,中断信号来自于输入线,通过边沿检测电路捕捉到后,如果中断屏蔽寄存器未屏蔽该中断,则信号输入中断挂起请求寄存器,然后传至 NVIC 中进行处理。

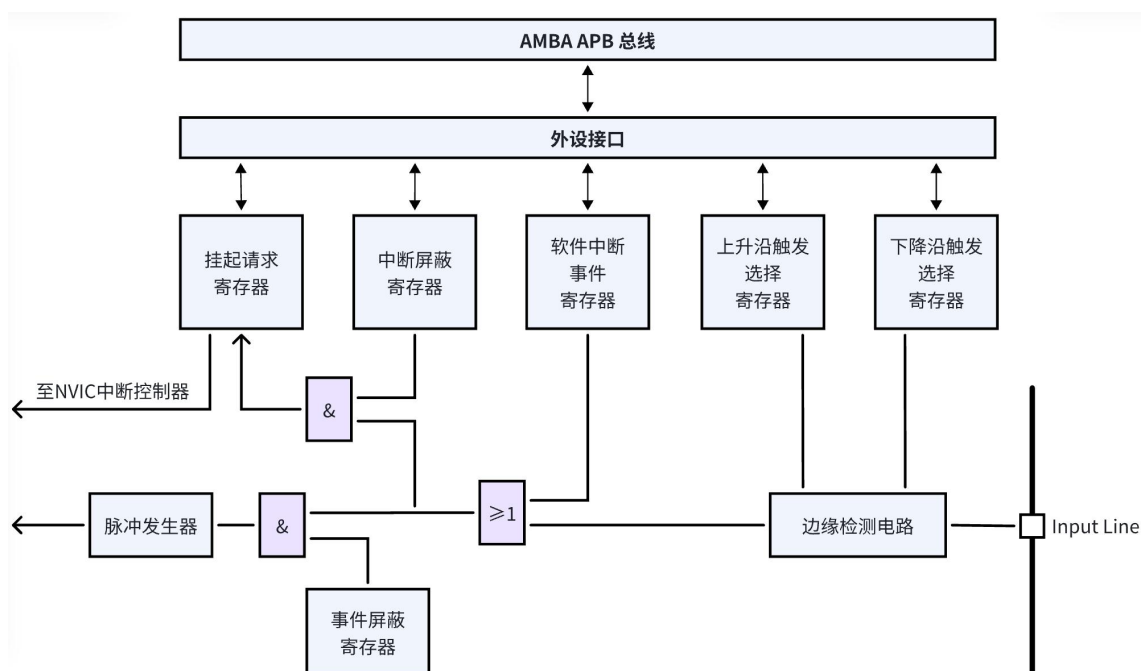


图 1-7 外部中断/事件控制器框图

EXIT 通过外部中断/事件线映射的方式,将 140 个 GPIO 口通过分组式方法链接到 EXIT 线上,用于收集中断信号。

3) 中断处理流程

中断输入和挂起：挂起即在处理中断 A 的时候，把中断 A 禁止了，然后在退出中断 A 的处理之前，又来了一个 A，则 A 自动被挂起，中断处于挂起状态，“等待处理”、“排队”，不会一直产生请求信号，或者处理中途请求撤销这种情况。中断入口处，多个寄存器会被压栈，同时，ISR 的起始地址会被从向量表中取出。处理完后，执行“异常返回”，之前压栈的寄存器纷纷出栈，被中断程序继续执行。中断活跃状态被清除。

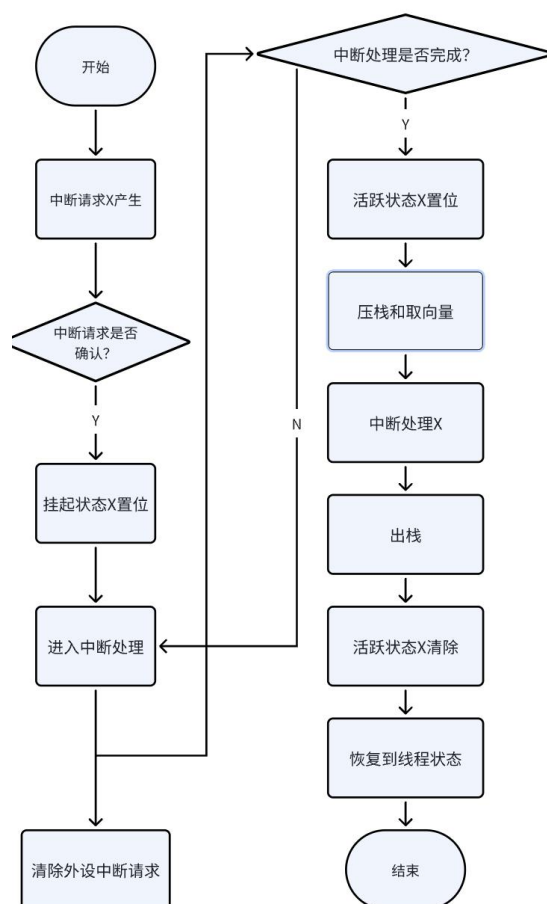


图 1-8 中断挂起和激活行为

1.2.4 ARM 内核时间驱动机制

根据 1.2.3 小节中的内容，我们已知中断是用于处理预设的突发事件，但上面讨论的事件是无规律的，只知道中断可能发生，而不知道何时发生。因此我们需要在此基础上引入计时器。其使得我们可以手动配置其频率（或者说时间间隔），以达到计时的功能。每过一段时间，就发起一次中断，进行对应的中断处理。基于此，我们可以周期性地执行一些特定的函数，便可以实现类似计时器的功能。

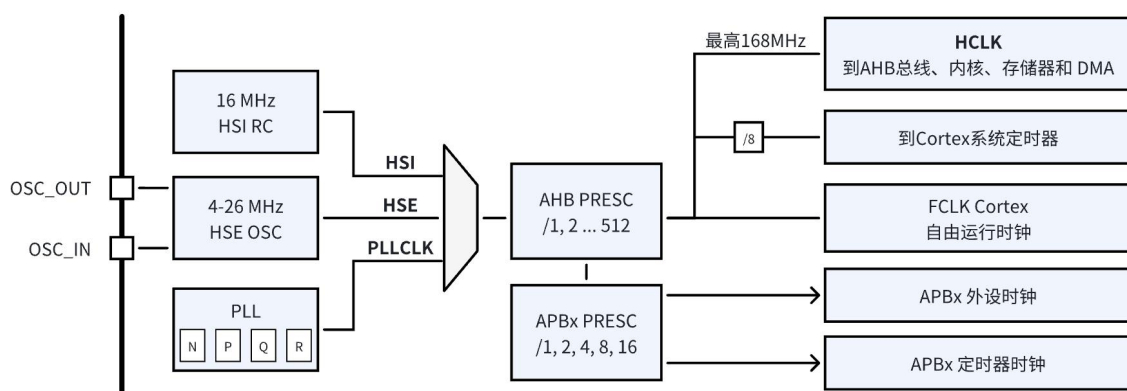


图 1-9 时钟树示意图

在具体实现中，我们首先要了解 STM32 的时钟频率是如何得到的。如图 1-11 所示，我们最后输出的总线频率 SYSCLK 依赖前面 SW 所选择的频率。SW 的选择有三种，HSI 为 RC 振荡器，精度不高，但上电自动启动，可用于执行最开始的代码，调节频率；HSE 为外部晶振，精度较高；PLLCLK 为锁相环时钟，可以用于调频，。在芯片开始启动时，先使用 HSI 的时钟，执行较慢。打开 HSE，并配好 PLL 的 P.Q.R.N 系数后，等待 PLL 输出到 SW，此时通过操作寄存器配置为选择输出便可以将此作为芯片运行时钟以及总线上的时钟了。

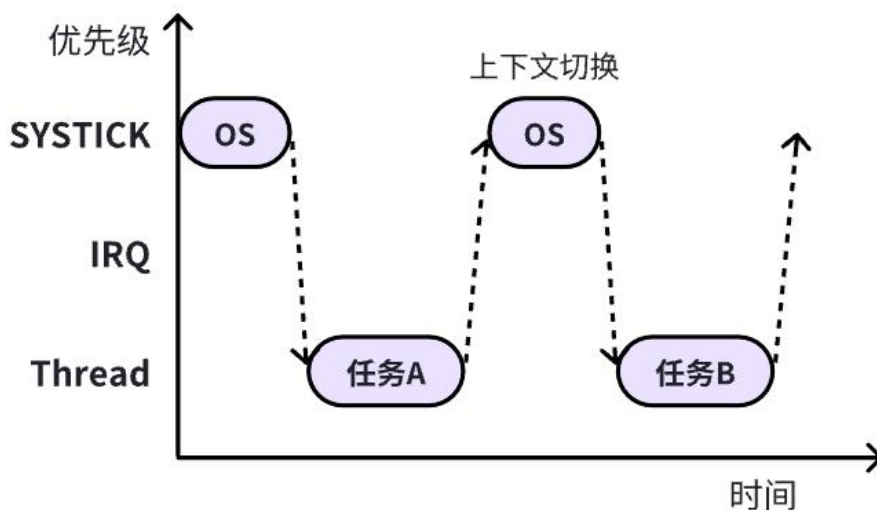


图 1-10 任务间通过 SysTick 轮转调度的简单模式

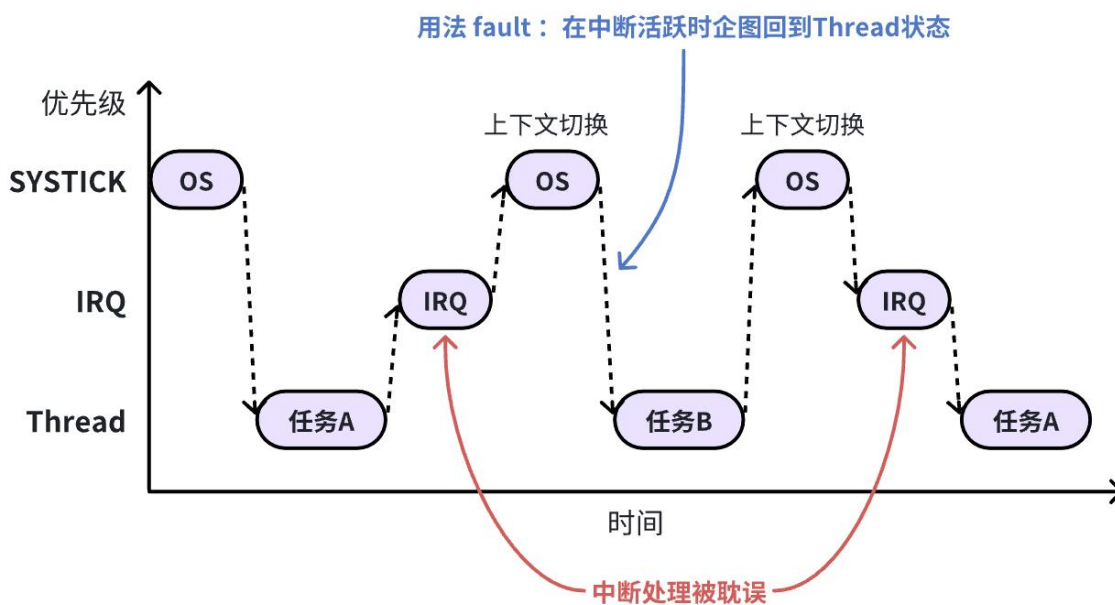


图 1-11 发生 IRQ 时上下文切换的问题

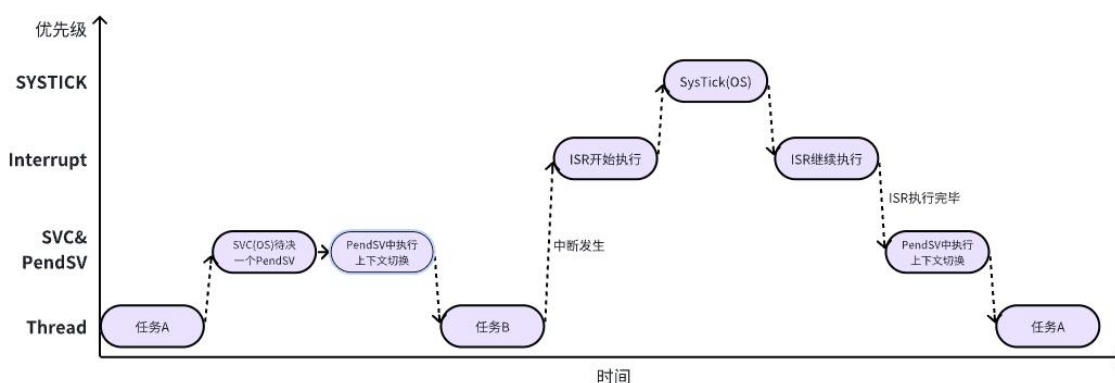


图 1-12 使用 PendSV 控制上下文切换

接下来需配置 STM32 单片机的心跳——SysTick，其时钟源于 AHB，频率与设定的 SW 输出频率一致。接着我们需要对 SK_CTRL 寄存器进行配置，选择不分频，设置计数器到 0 时产生中断请求，同时使能 SysTick。此外我们还需配置 STK_LOAD 寄存器，用于储存时基个数。SysTick 的周期等于时基个数乘以一个时基的时长（频率的倒数），根据计算得出的个数存入该寄存器。有了中断信号后，要编写中断服务函数，但此中断服务函数大部分不在 SysTick 中断中执行，而是在 pendSV 中，如图 1-13 所示，SysTick 是最高优先级任务，不能长时间停留，否则会耽误其他中断任务执行。所以在 SysTick 中完成必要操作后，要及时转到优先级较低的 pendSV 中，这样其他中断到来时才能做出响应，如图 1-14 所示，需要部分执行代码放在低优先级中，以便其他中断打断。

1.2.5 μ C/OS II 启动代码设计

实际开发过程中的启动分为两部分，第一部分是开发板的启动，通过中断向量表的 reset 中断实现；另一部分是 OS 的初始化和启动。OS 的初始化代码已经实现，我们只需重点关注开发板硬件的启动流程和配置，具体流程图如下图所示：



图 1-13 启动流程设计

μ C/OS II 系统有多个堆栈，包括一个系统堆栈和多个线程堆栈。故而在整个代码运行前需要先初始化这个系统堆栈，这样后面的系统才能够正常运行。在此处，我们需要手动的给芯片分配栈空间，并调整 SP 指针，供后续代码使用。

如在代码 1-3 L(1)代码表示 Stack_Size 大小为 0x400, L(2)表示此段存储空间为栈,可读可写,按 3 字节对齐。在 L(3)中,真正分配实际具体空间给标号 Stack_Mem。L(4)和 L(5)的作用同理,分配堆空间 0x200,可读可写,按 3 字节对齐,分配给标号 Heap_Mem 的变量。

代码 1-3 系统堆栈初始化

Stack_Size	EQU	0x00000400	(1)
	AREA	STACK, NOINIT, READWRITE, ALIGN=3	(2)
Stack_Mem	SPACE	Stack_Size	(3)
__initial_sp			
Heap_Size	EQU	0x00000200	(4)
	AREA	HEAP, NOINIT, READWRITE, ALIGN=3	
__heap_base			
Heap_Mem	SPACE	Heap_Size	(5)

重新上电时，由硬件控制，PC 会自动跳转到 Reset_Handler 执行复位中断函数，这里包含了 SystemInit 时钟频率配置和我们自行定义的分散加载文件 myScatterLoad。

代码 1-4 Reset_Handler 复位中断

Reset_Handler	PROC	
	EXPORT	Reset_Handler [WEAK]
IMPORT	SystemInit	
IMPORT	myScatterLoad	
IMPORT	main	

```

        LDR    R0, =SystemInit
        BLX    R0

        LDR    R0, =myScatterLoad
        BLX    R0

        BL     main
        B      .
    ENDP

```

在时钟频率配置部分，为了能增加计算能力，我们需要更改对应频率，也就是 1.2.4 章节中的修改 SYSCLK 的时钟源。为此，我们小组在 os_cpu_a.asm 文件中编写了 SystemInit 函数，用于修改时钟频率，在这个函数中，我们要实现的便是调配 PLL 锁相环时钟频率以及系统时基单元的初始化。

代码 1-5 SystemInit 时钟频率配置

```

SystemInit
    LDR R0, =RCC_CR
    LDR R1, [R0]           ; 读取当前 RCC_CR 的值
    ORR R1, R1, #(1 << 16) :or: (1 << 18)
    ; 若使用外部时钟源，添加: ORR R1, R1, #(1 << 18)
    STR R1, [R0]

Wait_HSE_Ready
    LDR R1, [R0]
    TST R1, #(1 << 17)
    BEQ Wait_HSE_Ready

    LDR R0, =FLASH_ACR
    MOV R1, #0x02
    STR R1, [R0]

    LDR R0, =RCC_PLLCFGR
    MOV R1, #0x00000000
    ORR R1, R1, #(8 << 0)      ; PLLM = 16 (bits 5:0)
    ORR R1, R1, #(336 << 6)   ; PLLN = 336 (bits 14:6)
    ORR R1, R1, #(1 << 16)    ; PLLP = 0b00
    ORR R1, R1, #(1 << 22)    ; PLLSRC = 1
    STR R1, [R0]

```

```
LDR R0, =RCC_CR
LDR R1, [R0]
ORR R1, R1, #(1 << 24)
STR R1, [R0]

Wait_PLL_Ready
LDR R1, [R0]
TST R1, #(1 << 25)
BEQ Wait_PLL_Ready

LDR R0, =RCC_CFGR
MOV R1, #0x00000000
ORR R1, R1, #(0 << 4) ; HPRE=0
ORR R1, R1, #(0x4 << 10) ; PPRE1=0b100
ORR R1, R1, #(0 << 13) ; PPRE2=0
ORR R1, R1, #(2 << 0) ; SW=0b10
STR R1, [R0]

Wait_Clock_Switch
LDR R1, [R0]
AND R2, R1, #0xC
CMP R2, #0x8
BNE Wait_Clock_Switch

LDR R0, =0xE000E010
MOV R1, #0x00000000
ORR R1, #5
STR R1, [R0]

LDR R0, =SYSTICK_BASE
LDR R1, =83999
STR R1, [R0, #0x04]
MOV R1, #0
STR R1, [R0, #0x08]
MOV R1, #0x07
STR R1, [R0, #0x00]

LDR R0, =SCB_SHPR3
ORR R1, #(0x5 << 24)
STR R1, [R0]
```



```

CPSIE I
BX LR

ALIGN
END

```

时钟初始化过后就是代码拷贝和清除，在此部分我们设计代码拷贝为 4 字节对齐的拷贝规则，实现 DATA 段复制，而清除 BSS 段则选择自上而下 4 字节对齐的清除方式。

代码 1-6 myScatterload 分散加载和清除

```

AREA |.text|,CODE,READONLY
myScatterLoad PROC
    EXPORT myScatterLoad
    IMPORT |Image$$RW_IRAM1$$Base|
    IMPORT |Image$$RW_IRAM1$$Length|
    IMPORT |Load$$RW_IRAM1$$Base|
    IMPORT |Image$$RW_IRAM1$$ZI$$Base|
    IMPORT |Image$$RW_IRAM1$$ZI$$Length|

    ; 复制数据段
    LDR R0, = |Load$$RW_IRAM1$$Base|
    LDR R1, = |Image$$RW_IRAM1$$Base|
    LDR R2, = |Image$$RW_IRAM1$$Length|
CopyData
    SUB R2, R2, #4
    LDR R3, [R0, R2]
    STR R3, [R1, R2]
    CMP R2, #0
    BNE CopyData

    ; 清除 BSS 段
    LDR R0, = |Image$$RW_IRAM1$$ZI$$Base|
    LDR R1, = |Image$$RW_IRAM1$$ZI$$Length|
CleanBss
    SUB R1, R1, #4
    MOV R3, #0
    STR R3, [R0, R1]
    CMP R1, #0
    BNE CleanBss
    ALIGN

```

ENDP

END

完成以上步骤，系统进入我们编写的 main 函数，启动 $\mu C/OS$ 以及 SysView 后开始执行相应的多个任务。

代码 1-7 main 函数编写

```
int main(void) {
    OSInit();

    SEGGER_SYSVIEW_Conf();
    SEGGER_SYSVIEW_Start();

    OSStart();
}
```

1.2.6 $\mu C/OS$ II 内核移植设计

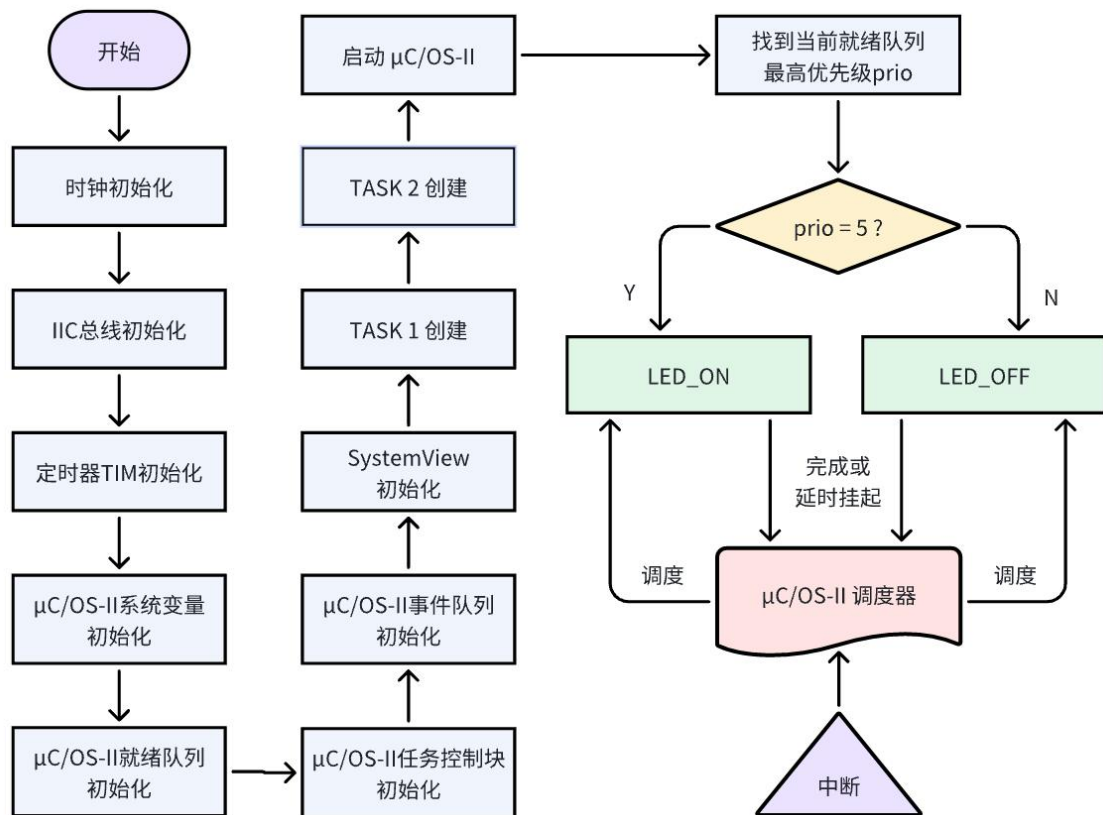


图 1-14 操作系统移植业务流程图

我们设置了两个任务用来验证我们的操作系统移植，首先我们的代码初始化时钟、定时器以及 μ C/OS-II 的一些基本配置，然后创建这两个任务（LED_ON 和 LED_OFF）并启动。开始运行后，系统会调用 OS_SchedNew() 函数查找就绪任务中优先级最高的任务，由调度器调度该任务执行，当任务完成或演示挂起后，调度任务会调度另一优先级最高的任务执行，在此过程中，OSTimeDly 主动挂起后再由操作系统调用 PendSV 中断，调度另一个任务执行，最终实现板载 LED 灯闪烁，初步证明操作系统能够成功移植并完成多任务并发。

1.2.7 SystemView 模块移植设计

SystemView 是由 SEGGER 公司开发的一款嵌入式系统可视化分析工具，常常被用于嵌入式系统开发和调试。它提供了一个全面的监控系统，帮助开发人员深入了解系统的行为和性能。SystemView 通过在系统中插入轻量级的事件记录器，实时捕获和记录系统中发生的事件，如任务切换、中断、函数调用等。同时，开发人员可以通过可视化的界面实时观察系统的运行情况，包括任务的执行顺序、任务的运行时间、中断的发生和处理时间等。将 SystemView 运用于 μ c-osii 操作系统上，有助于我们更好地理解 and 调试嵌入式系统，提高四轴飞行器系统的性能和稳定性。

SystemView 有两个主要构成部分：一是 PC 端的可执行文件，二是与目标硬件代码相关的库和程序源码，其应用架构如下图所示。

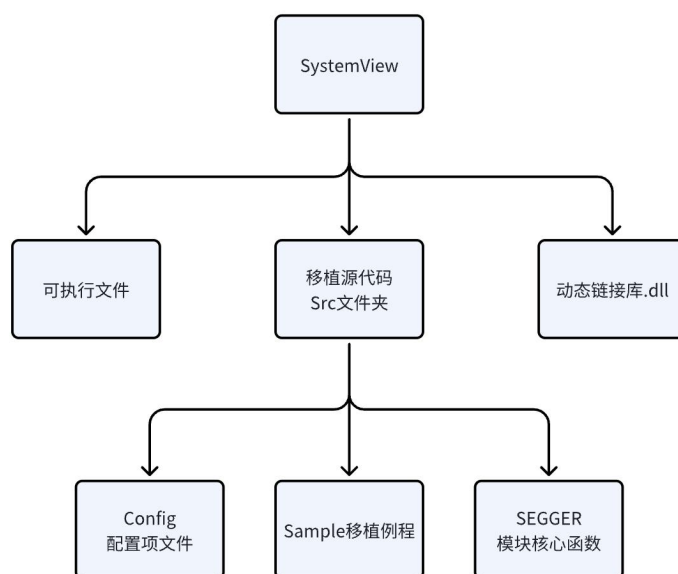


图 1-15 SytemView 应用移植架构图

为了将其运用于项目工程中，我们重点关注上图的"移植源代码 Src 文件夹"。首先要将官方例程加入项目中；其次，对 SystemView 中的参数根据 μcosii 操作系统进行自定义配置；最后在操作系统代码引入 SystemView 接口，即可成功搭建 SystemView 测试环境。后面再烧录 KEIL 项目代码，启动 SystemView 应用程序，即可执行系统监控。对应移植设计的整体流程图如下所示。

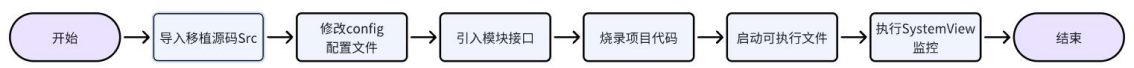


图 1-16 SytemView 移植设计的整体流程图

1.3 针对复杂工程问题的方案实现

1.3.1 用 KEIL 重构 $\mu\text{C}/\text{OS II}$ 新工程

首先，我们需要在 keil 中添加 $\mu\text{C}/\text{OS-II}$ 源代码，构建 `os_cpu.h` 等与处理相关代码文件，并顺利通过编译。

1) 项目框架构建

新建一个文件夹 $\mu\text{C}/\text{OS II}$ ，并在文件夹下再新增三个文件夹 Core、Port 和 Cfg。

os.h	2025/3/5 16:18	C Header 源文件	2 KB
os_core.c	2025/4/8 16:19	C 源文件	87 KB
os_dbg_r.c	2025/3/5 16:18	C 源文件	13 KB
os_flag.c	2025/3/5 16:18	C 源文件	56 KB
os_mbox.c	2025/3/5 16:18	C 源文件	32 KB
os_mem.c	2025/3/5 16:18	C 源文件	20 KB
os_mutex.c	2025/3/5 16:18	C 源文件	40 KB
os_q.c	2025/3/5 16:18	C 源文件	43 KB
os_sem.c	2025/4/8 18:35	C 源文件	30 KB
os_task.c	2025/3/5 16:18	C 源文件	60 KB
os_time.c	2025/3/5 16:18	C 源文件	11 KB
os_tmr.c	2025/3/5 16:18	C 源文件	45 KB
os_trace.h	2025/3/5 16:18	C Header 源文件	11 KB
ucos_ii.c	2025/3/5 16:18	C 源文件	2 KB
ucos_ii.h	2025/3/5 16:18	C Header 源文件	79 KB

图 1-17 $\mu\text{C}/\text{OS II}$ 操作系统 Core 代码

CORE 文件夹包含了 μ C/OS II 内核的核心功能模块。这些模块提供了任务管理、时间管理、资源管理、任务通信和同步等核心功能，是 μ C/OS II 内核的基础。

 os_cpu.h	2025/3/5 16:17	C Header 源文件	11 KB
 os_cpu_a.asm	2025/3/5 16:17	Assembler Source	17 KB
 os_cpu_c.c	2025/3/9 18:41	C 源文件	36 KB
 os_dbg.c	2025/3/5 16:17	C 源文件	14 KB

图 1-18 μ C/OS II 操作系统 Ports 代码

Ports 文件夹中的文件包含了针对不同处理器平台的移植代码。每个处理器平台都有不同的硬件架构和操作系统接口，因此需要根据处理器的特点进行移植，以确保 μ C/OS II 内核可以在特定处理器上正确地运行。

 os_cfg.h	2025/3/5 16:17	C Header 源文件	11 KB
--	----------------	--------------	-------

图 1-19 μ C/OS II 操作系统 Cfg 代码

CONFIG 文件夹中的文件用于配置 μ C/OS II 内核的各种功能和选项。CORE、PORT 和 CONFIG 是 μ C/OS II 内核的三大模块，通过这些模块的工作协同，我们可以根据目标系统的需求定制和优化 μ C/OS II 内核，使其在 STM32F401 开发板上高效运行。

2) 项目导入

打开 keil 5，把它们导入到项目中。选择 project -> new uversion project，然后选好芯片型号：STM32F401RETx。

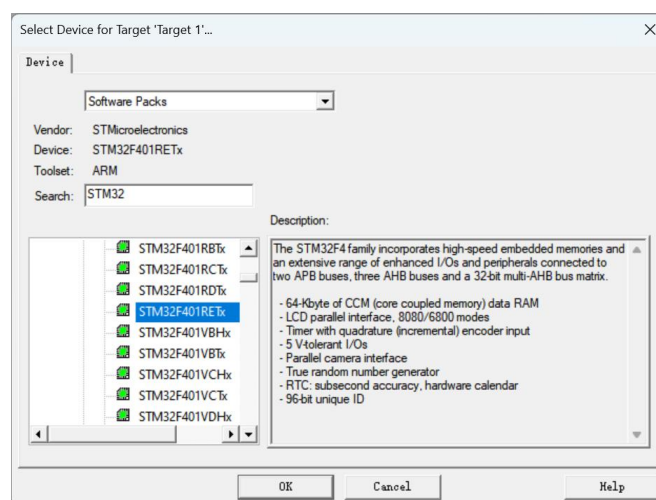


图 1-20 新建工程中芯片选择页面

3) 右键单击 target 1-add group 新建文件夹，并重命名为 Core、Port 和 Cfg，右键单击文件夹，add existing files to group ‘xxx’ 添加文件。

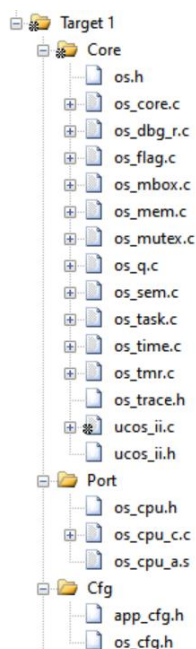


图 1-21 为 keil 中文件夹添加需要编译的文件

4) 添加头文件路径。打开魔术棒，在 C/C++和 Asm 里的 Include Path 中写入 C 程序和汇编文件的路径。

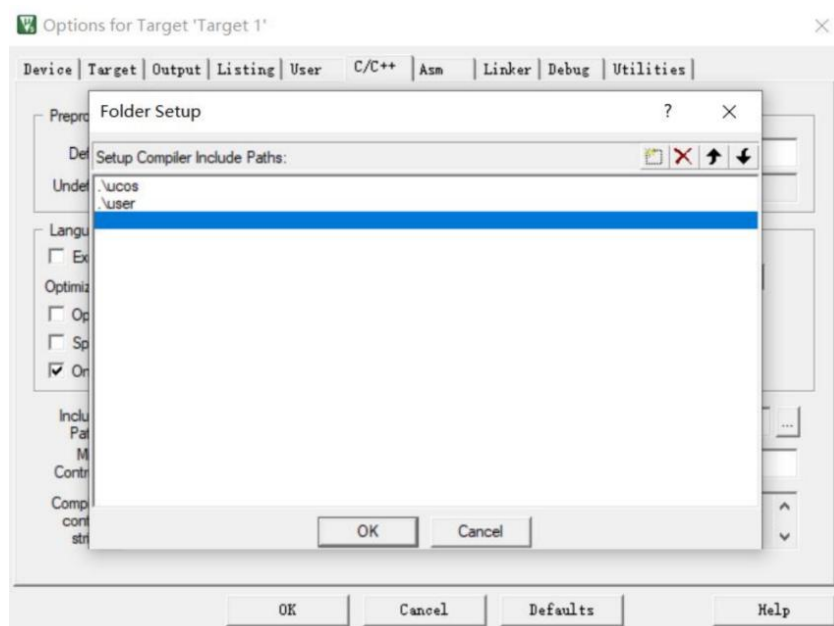


图 1-22 添加头文件所在路径

5) 解决若干编译报错

① 缺少文件或者找不到文件。

.\\ucos\\ucos_ii.h(45): error: #5: cannot open source input file "xxx": No such file or directory

解决方案：手动在 ucoss 中添加相应文件。

② INTxxU 型变量未定义

.\\ucos\\ucos_ii.h(1207): error: #20: identifier "INTxxU" is undefined

解决方案：在 os_cpu.h 中用 typedef 定义相应变量。32 位单片机上，32U 对应 unsigned int，16U 对应 unsigned short，8U 对应 unsigned char。

代码 1-7 main 函数编写

```
typedef unsigned char INT8U;  
typedef unsigned int INT32U;  
typedef unsigned short INT16U;
```

③ 由非 INTxxxU 型变量未定义引发的错误：

.\\ucos\\ucos_ii.h(719): error: #20: identifier "xxx" is undefined

解决方案：在 os_cfg.h 里，用#define 补上所需的宏定义

代码 1-8 增添宏定义相关参数值

```
#define OS_MAX_TASKS 64  
#define OS_LOWEST_PRIO 64  
#define OS_TASK_IDLE_STK_SIZE 64
```

④ 宏定义中的变量未定义，具体解决办法和错误类型 4 相同。

.\\ucos\\ucos_ii.h(1482): error: #35: #error directive: "OS_CFG.H, Missing OS_FLAG_EN: Enable (1) or Disable (0) code generation for Event Flags"

代码 1-9 在 os_cfg.h 中添加的宏定义

```
#define OS_FLAG_EN 0
```

⑤ 因为找不到指定函数导致链接错误：

.\\Objects\\baogao.axf: Error: L6218E: Undefined symbol

解决方案：查看报错中缺少的函数并添加。因为函数的声明和定义的语法规

则 区别很大，编译器可以轻松地区分什么是声明，什么是定义。如果分别编译单个文件时，只有声明没有定义，编译器暂时不会报错，会尝试在链接时去其他文件中找相应的函数定义，但是如果链接时还没有找到函数体，编译器就会报错。新建一个 `os_cpu.c`，补充缺少的函数定义。

代码 1-10 `os_cpu.c` 中与处理器相关的代码

```
#include "os_cpu.h"
void SystemInit(){}
void OSInitHookBegin(){}
void OSInitHookEnd(){}
void OSTCBInitHook(){}
void OSTaskCreateHook(){}
void OSTaskIdleHook(){}
void OSTaskReturnHook(){}
OS_STK *OSTaskStkInit(void (*task)(void *p_arg),
                      void *p_arg, OS_STK
                      *ptos, INT16U opt){
    return ptos;
}
void OSTaskSwHook(){}
void OS_CPU_ExceptStkBase(){}
void OS_KA_BASEPRI_Boundary(){}

```

⑥ 临界区函数未定义错误：

.\Objects\baogao.axf: Error: L6218E: Undefined symbol OS_ENTER_CRITICAL (referred from ucos_ii.o).

.\Objects\baogao.axf: Error: L6218E: Undefined symbol OS_EXIT_CRITICAL (referred from ucos_ii.o).

warning: #223-D: function "OS_EXIT_CRITICAL" declared implicitly

解决方案：在 `os_cfg.h` 中加上临界区保护函数：

代码 1-11 `os_cfg.h` 中临界区保护相关代码

```
extern OS_CPU_SR;
//在 os_cfg.h 中定义为 unsigned char 类型。
extern OS_CPU_SR OS_CPU_SR_Save(void);
//引用汇编文件 os_cpu_a.asm 中的函数
extern void OS_CPU_SR_Restore(OS_CPU_SR);
//引用汇编文件 os_cpu_a.asm 中的函数

```



```
extern void OSCtxSw(void);
//引用汇编文件 os_cpu_a.asm 中的函数
#define OS_ENTER_CRITICAL() (cpu_sr = OS_CPU_SR_Save())
#define OS_EXIT_CRITICAL() (OS_CPU_SR_Restore(cpu_sr))
#define OS_CRITICAL_METHOD 3u
//表示采用第三种临界区保护方式
```

⑦ 编译文件并修改相关警告

.\ucos\os_core.c(723): warning: #223-D: function "OSIntCtxSw" declared implicitly

.\ucos\os_core.c(876): warning:#223-D: function "OSStartHighRdy" declared implicitly

解决方案：这里需要使用到 `extern` 关键字。这两个函数名标号在其他文件中定义，而在 `os_core.c` 中没有定义，故需使用 `extern` 关键字。

代码 1-12 `os_core.c` 中补充 `extern` 关键字部分代码

```
extern void OSIntCtxSw(void);
extern void OSStartHighRdy(void);
```

最终实现了 0 error 0 warning。

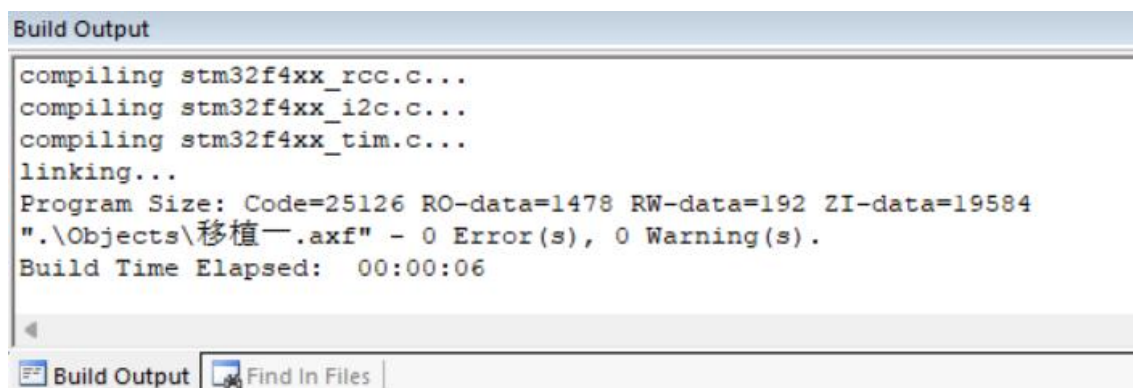


图 1-23 编译结果图

1.3.2 事件驱动实现

1.3.2.1 任务堆栈初始化

堆栈是一种用来保存任务现场的数据结构，任务在运行时必须有一个完整的

堆栈。如果堆栈没有被正确地初始化，会导致任务运行时出现各种异常。因此，在任务初始化时，需要对任务的堆栈进行正确的初始化。

堆栈在内存中被模拟成一块区域，主要用于保存 CPU 的所有寄存器和任务的入口函数地址。当操作系统要建立一个任务时，首先需要对任务的堆栈空间进行初始化。在 $\mu\text{C}/\text{OS-II}$ 中，任务的堆栈初始化主要通过 `OSTaskStkInit()` 函数实现，其函数的原型为：`OS_STK *OSTaskStkInit(void (task)(voidp_arg), void *p_arg, OS_STK *ptos, INT16U opt)`。其中，`task` 参数是任务执行函数的入口地址，`p_arg` 是任务参数的地址，`ptos` 是堆栈顶部指针，`opt` 是预留参数，函数最终返回初始化后的堆栈指针。

在堆栈初始化过程中，需要保存任务的起始地址、处理器状态字、`EXC_RETURN` 参数和一些寄存器等信息。根据 STM32 的寄存器结构，堆栈初始化函数需要按照特定的顺序保存寄存器的值，以保证运行环境能够正确恢复。因此，在进行上下文切换时，需要严格按照模拟压栈的顺序来保存和恢复寄存器的值。

代码 1-13 OSTaskStkInit 函数

```
OS_STK *OSTaskStkInit(void (*task)(void *p_arg),
                      void *p_arg, OS_STK *ptos,
                      INT16U opt) {
    OS_STK *stk;
    opt = opt;
    stk = (OS_STK *)((OS_STK)(ptos) & 0xFFFFFFF8u); // 双字对齐
    *--stk = (OS_STK)0x01000000uL; // xPSR 寄存器
    *--stk = (OS_STK)task; // PC 寄存器，指向任务函数的地址
    *--stk = (OS_STK)0xFFFFFFFEL; // LR 寄存器，设置为异常返回地址
    *--stk = (OS_STK)0x12; // R12 寄存器
    *--stk = (OS_STK)0x3; // R3 寄存器
    *--stk = (OS_STK)0x2; // R2 寄存器
    *--stk = (OS_STK)0x1; // R1 寄存器
    *--stk = (OS_STK)p_arg; // R0 寄存器，任务函数的参数
    *--stk = (OS_STK)0xFFFFFFF0; // EXEC_RETURN 寄存器，异常返回值
    *--stk = (OS_STK)0x11; // R11 寄存器
    *--stk = (OS_STK)0x10; // R10 寄存器
    *--stk = (OS_STK)0x9; // R9 寄存器
    *--stk = (OS_STK)0x8; // R8 寄存器
    *--stk = (OS_STK)0x7; // R7 寄存器
    *--stk = (OS_STK)0x6; // R6 寄存器
```

```

*(--stk) = (OS_STK)0x5;           // R5 寄存器
*(--stk) = (OS_STK)0x4;           // R4 寄存器
return stk;                        // 返回栈指针
}

```

如代码 1-12 所示，在 OSTaskStkInit 函数中我们首先通过对任务堆栈指针进行位操作，确保堆栈地址按照 AAPCS 规则进行了双字对齐，这样可以保证在使用堆栈时不会发生错误。在 STM32 任务切换时，硬件会将 xPSR、PC、LR、R12、R0-R3 寄存器自动入栈，即按照硬件寄存器入栈顺序进行寄存器保存。后续将 R11-R4 通用寄存器顺序入栈，最终返回最新的 stk 指针。

1.3.2.2 任务上下文切换

μC/OS-II 中，任务上下文切换是实现任务切换的关键步骤之一。当一个任务需要释放 CPU 的使用权时，系统必须保存当前任务的执行环境，并将 CPU 的控制权交给下一个待执行的任务。任务的上下文切换发生在如下情况中：

- 外部中断导致更高优先级就绪，触发上下文切换；
- SysTick 使更高优先级的延时任务的延时到期，触发上下文切换；
- 任务主动调用如 OSTimeDly 等挂起自己的函数，触发上下文切换。

其中，前两种由 OSIntExit 调用 OSIntCtxSw 实现上下文切换，也就是中断引发的被动上下文切换；而第三种则调用 OSCtxSw 实现上下文切换，称之为主动上下文切换。与 S3C2440 基于 ARM9 内核不同的是，我们使用的开发板 STM32F401RE 是基于 ARM Cortex-M4 内核。

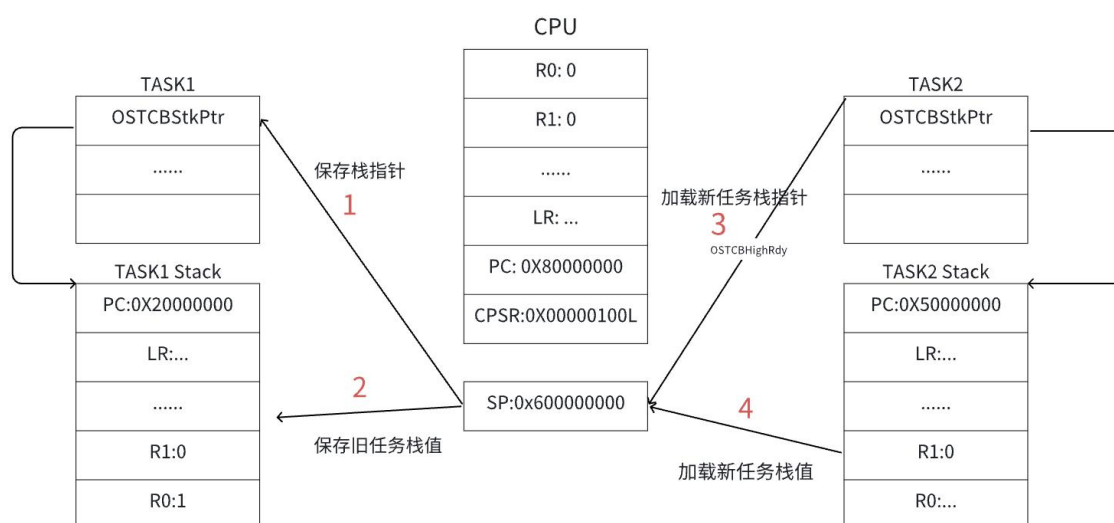


图 1-24 任务上下文切换示意图

如上图所示，任务的上下文切换可以通过以下四个步骤来进行：

① 保存当前任务的执行环境：在进行任务切换之前，需要将当前任务的执行环境保存起来，以便稍后能够正确地恢复。这包括保存寄存器的状态和堆栈信息等。

② 更新当前任务的堆栈指针：在保存完当前任务的执行环境后，需要将新的堆栈指针保存到当前任务的 TCB（任务控制块）中。TCB 通常包含了任务的各种状态信息，包括堆栈指针。

③ 设置新任务的堆栈指针：接下来，将被抢占任务的堆栈地址（通常保存在 OSTCBHighRdy->OSTCBStkPtr 中）赋给堆栈指针，以便从该位置恢复新任务的执行环境。

④ 恢复新任务的执行环境：最后，根据新任务的执行环境，在新任务的堆栈指针位置进行恢复。这包括恢复寄存器的状态、堆栈信息以及其他必要的执行环境。

代码 1-14 OSCtxSw 与 OSIntCtxSw 函数

```

;OSCtxSw/OSIntCtxSw
; 保存当前任务的执行环境，并切换到下一个待执行的任务
OSCtxSw:
    ldr r0, =NVIC_ICS_REG ; 将 NVIC_ICS_REG 的地址加载到寄存器 r0 (1)
    ldr r1, =NVIC_PENDSV_SETPEND
    ; 将 NVIC_PENDSV_SETPEND 的地址加载到寄存器 r1
    ldr r2, [r0] ; 将 NVIC_ICS_REG 中的值加载到寄存器 r2 (2)
    orr r2, r1, r2 ; 将 r1 和 r2 进行或运算，并将结果存储回 r2
    str r2, [r0] ; 将寄存器 r2 中的值存储回 NVIC_ICS_REG
    bx lr ; 函数返回

```

在 OSCtxSw() 函数与 OSIntCtxSw() 函数实现中，两者采用了一致的实现方法。L(1) 实现触发 PendSV 异常，L(2) 实现向 NVIC_ICS_REG 写入 NVIC_PENDSV_SETPEND 触发 PendSV 中断。在函数实现中，我们可以看到函数实际上是触发了 PendSV 中断，再由 PendSV 中断例程完成真正的任务切换。

实际上，S3C2440 的上下文切换是直接在 OSCtxSw 或者 OSIntCtxSw 中完成的。而我们开发使用的 STM32F4 是 Cortex-M 系列，实际的上下文切换都是在 PendSV_Handler 中完成的。

结合我们前文的三种上下文切换场景，外部中断退出时，调用 OSIntExit 函数会激活 PendSV 中断，而真正的任务切换是在 PendSV 中断例程中实现的；同

样地，SysTick 中断优先级较高，并不直接进行任务切换，而是在中断退出时调用 OSIntExit 函数来激活 PendSV 中断；任务切换时调用 OS_Sched，该函数通过 OS_TASK_SW 调用 OSCtxSw 来激活 PendSV 中断，进而由 PendSV 中断例程完成真正的任务切换。

总结下来就是 μ C/OS-II 中所有的任务切换最终都是由 PendSV 中断例程完成的，而且 PendSV 的中断优先级必须是系统所有中断中最低的。

代码 1-15 PendSV_Handler 函数实现

```
PendSV_Handler
; 保存旧任务的上下文寄存器，包括 r4-r11 和 lr
mrs r0, psp ; 获取旧任务的 psp
stmfd r0!, {r4-r11, lr}
; 保存旧任务的其余寄存器。lr 将自动更新为 EXEC_RETURN

; 将旧任务的 PSP 保存到 OSTCBCur->OSTCBStkPtr 中
ldr r1, =OSTCBCur
ldr r2, [r1]
str r0, [r2]
mov r4, lr ; 保存 lr，以防执行 OSTaskSwHook

; 调用 OSTaskSwHook
; BL OSTaskSwHook
; 通过设置 EXEC_RETURN 返回到进程堆栈
orr lr, r4, #0x04

; 更新 OSPrioCur 为 OSPrioHighRdy
ldr r0, =OSPrioCur ; OSPrioCur = OSPrioHighRdy
ldr r1, =OSPrioHighRdy
ldrb r2, [r1]
ldrb r2, [r0]

; 更新 OSTCBCur 为 OSTCBHighRdy
ldr r0, =OSTCBHighRdy ; OSTCBCur = OSTCBHighRdy
ldr r1, =OSTCBCur
ldr r0, [r0]
str r0, [r1]

; SYSVIEW 任务切换事件
push {r0, r1}
```

```

mov r0, r1
bl SYSVIEW_TaskSwitchedIn
pop {r0, r1}

; 恢复新任务的寄存器
ldr r1, [r0] ; r1 = sp (新任务的 psp)
ldmfd r1!, {r4-r11, lr} ; 恢复新任务的寄存器
msr psp, r1 ; 将 psp 的值设为新任务的 sp

bx lr ; 从中断返回

```

如上述代码所示，该代码是用于处理任务切换的 PendSV_Handler 中断服务程序。首先，它保存旧任务的上下文寄存器，包括 r4-r11 和 lr，将其存储在旧任务的 PSP 所指向的堆栈中。然后，将旧任务的 PSP 值保存到 OSTCBCur -> OSTCBStkPtr 中。接下来，保存 lr 寄存器，并执行了一个可选的钩子函数 OSTaskSwHook。通过设置 lr 的 EXEC_RETURN 位为 1 来返回到新任务的堆栈。然后，更新 OSPrioCur 为 OSPrioHighRdy，以及更新 OSTCBCur 为 OSTCBHighRdy。接下来，执行了一个 SYSVIEW 任务切换事件。然后，恢复新任务的寄存器，将 psp 的值设置为新任务的 sp，最后从中断中返回。

1.3.3 时间驱动实现

1.3.3.1 时钟配置

我们组选择在 os_cpu.s 文件里实现系统时钟初始化，主要原因是，保证在系统启动时，会在 os_cpu.s 中执行系统调用函数，将 system_Init 函数配置在其中，可以在启动阶段的部分完成时钟初始化，保证各任务顺利运行。



图 1-25 系统时钟配置流程图

代码 1-16 时钟初始化

```

SystemInit
    LDR R0, =RCC_CR //选择时钟源
    LDR R1, [R0]
    ORR R1, R1, #(1 << 16) :or: (1 << 18)
    STR R1, [R0]

```

Wait_HSE_Ready

```
LDR R1, [R0]    //选择 HSE 作为系统时钟源
TST R1, #(1 << 17)
BEQ Wait_HSE_Ready
LDR R0, =FLASH_ACR    //配置 ACR 寄存器
MOV R1, #0x02
STR R1, [R0]

LDR R0, =RCC_PLLCFGR    //配置锁相环 PLL
MOV R1, #0x00000000
ORR R1, R1, #(8 << 0)    ; PLLM = 16 (bits 5:0)
ORR R1, R1, #(336 << 6)    ; PLLN = 336 (bits 14:6)
ORR R1, R1, #(1 << 16)    ; PLLP = 0b00
ORR R1, R1, #(1 << 22)    ; PLLSRC = 1
STR R1, [R0]

LDR R0, =RCC_CR
LDR R1, [R0]
ORR R1, R1, #(1 << 24)    //配置系统时钟模式为锁相环
STR R1, [R0]
```

Wait_PLL_Ready

```
LDR R1, [R0]
TST R1, #(1 << 25)
BEQ Wait_PLL_Ready

LDR R0, =RCC_CFGR
MOV R1, #0x00000000
ORR R1, R1, #(0 << 4)    ; HPRE=0
ORR R1, R1, #(0x4 << 10)    ; PPRE1=0b100
ORR R1, R1, #(0 << 13)    ; PPRE2=0
ORR R1, R1, #(2 << 0)    ; SW=0b10
STR R1, [R0]
```

Wait_Clock_Switch

```
LDR R1, [R0]
AND R2, R1, #0xC
CMP R2, #0x8
BNE Wait_Clock_Switch
```

在完成初始化后我们正式进入配置部分，我们主要针对系统时钟的时基单元包括 ARR、CCR 等值进行配置

代码 1-17 时基单元部分

```
LDR R0, =0xE000E010
MOV R1, #0x00000000
ORR R1, #5
STR R1, [R0]
LDR R0, =SYSTICK_BASE
LDR R1, =83999
STR R1, [R0, #0x04]
MOV R1, #0
STR R1, [R0, #0x08]
MOV R1, #0x07
STR R1, [R0, #0x00]
LDR R0, =SCB_SHPR3
ORR R1, #(0x5 << 24)
STR R1, [R0]
```

1.3.3.2 SysTick 中断配置

Systick 定时器就是系统滴答定时器，一个 24 位的倒数（从大到小）定时器，计到 0 时，将从 RELOAD 寄存器中自动重装载定时初值。移植 μ C/OS-II 需要使用 Systick 来作为系统时钟。配置 SysTICK 主要通过 My_Systick_Config() 和 SysTick_Handler() 两个函数来完成。

代码 1-18 My_Systick_Config() 函数实现

```
void My_Systick_Config(uint32_t reload_value){
    SysTick->LOAD |= reload_value-1;
    SysTick->VAL |= SysTick->LOAD;
    SysTick->CTRL |= 1<<2;
    SysTick->CTRL |= 1<<1;
    SysTick->CTRL |= 0x01;
}
```

代码 1-19 SysTick_Handler()函数实现

```
SysTick_Handler
; OS_SaveSR
cpsid i
mrs r0, BASEPRI          ; save previous BASEPRI
mov r1, #0                ; set new BASEPRI
```



```
msr BASEPRI, r1
dsb
isb
cpsie i

; OSIntEnter
push {lr}
bl OSIntEnter
pop {lr}

; OS_RestoreSR
cpsid i
msr BASEPRI, r0
dsb
isb
cpsie i

; if OSIntNesting == 1
ldr r1, =OSIntNesting
ldrb r2, [r1]          ; OSIntNesting : int8u
cmp r2, #1
beq OSTickISR_FromTask
bne OSTickISR_FromInt

OSTickISR_FromTask
; save remained registers & save psp
mrs r0, psp
stmfd r0!, {r4-r11, lr}
msr psp, r0
ldr r2, =OSTCBCur
ldr r3, [r2]
str r0, [r3]
bl OSTimeTick
bl OSIntExit

; restore remained registers
mrs r0, psp
ldmfd r0!, {r4-r11, lr}
msr psp, r0
bx lr
```

```

OSTickISR_FromInt
    ; save remained registers
    mrs r0, msp
    stmfd r0!, {r4-r11, lr}
    msr msp, r0
    bl OStimeTick
    bl OSIntExit

    ; restore remained registers
    mrs r0, msp
    ldmdfd r0!, {r4-r11, lr}
    msr msp, r0
    bx lr

```

My_Systick_Config 函数的作用就是重载值初始化和配置 SysTick 定时器，将时钟源设置为 AHB，启用中断，并启动计数器。SysTick_Handler 函数的作用是保存和恢复 BASEPRI 寄存器，调用 OSIntEnter 函数，以及检查 OSIntNesting 的值以确定执行路径，如果 OSIntNesting 等于 1，意味着 SysTick 中断发生时，系统已经在为一个中断服务。在这种情况下，程序会调用 STickISR_FromTask，在其中会进行额外的保存原始任务的堆栈指针的 PSP 工作；否则它将调用 OSTickISR_FromInt，确保整个函数在 MSP 的操纵下执行。

1.3.4 μ C/OS II 在 STM-32 上的移植实现

在前面的项目中我们已经实现了事件驱动、时钟配置、启动文件、上下文切换，接下来，我们只需要完善系统的运行逻辑即可。在系统启动之初，需要有函数，能自发调用第一个任务，才能让接下来的调度和上下文切换正常运行，我们在 os_cpu.s 中编写了一个 OSStartHighRdy 函数，用于系统开始时的首次调度，此时不需要进行旧任务的保存工作，只需要获取最高优先级且就绪的任务，将其栈数据更新至 cpu 即可。

代码 1-20 OSStartHighRdy 函数实现

```

OSStartHighRdy
    CPSID    I
    STRB     R1, [R0]

    MOVN     R0, #0

```

```

MSR    PSP, R0

LDR    R0, =OS_CPU_ExceptStkBase
MSR    MSP, R1

BL     OSTaskSwHook
LDR    R0, =OSRunning
MOVS   R1, #1
STRB   R1, [R0]

LDR    R0, =OSPrioCur
LDR    R1, =OSPrioHighRdy
LDRB   R2, [R1]
STRB   R2, [R0]

LDR    R0, =OSTCBCur
LDR    R1, =OSTCBHighRdy
LDR    R2, [R1]
STR    R2, [R0]

LDR    R0, [R2]
MSR    PSP, R0

MRS    R0, CONTROL
ORR    R0, R0, #2
MSR    CONTROL, R0
ISB

LDMFD  SP!, {R4-R11, LR}
LDMFD  SP!, {R0-R3}
LDMFD  SP!, {R12, LR}
LDMFD  SP!, {R1, R2}
CPSIE  I
BX     R1

```

根据我们的系统逻辑设计图，我们，接下来只需要完成系统初始化，并创建两个任务（优先级分别为 5 和 6）并设置一个外部中断。其中优先级 5 的任务为电机驱动任务，用于将中断时接收机接收到的数据，处理并配置成电机速率使其转动；优先级 6 的任务为获取姿态传感器的数据，并通过蓝牙上传至手机端。

首先是 main 函数的编写，主要完成对功能函数和 OS 系统的初始化，并创建

任务，系统会自动调用 OSStartHighRdy 完成调度。

代码 1-21 main 函数

```
int main(void){
    //功能函数初始化
    PPM_Init();
    PWM_Init();
    GY86_Init();
    BLE_Init();
    OLED_Init();
    OSInit();    //操作系统初始化
    //创建任务
    OSTaskCreate(TASK_ChangeMotor, (void*)0,
                (OS_STK*)&TASK_ChangeMotorstk
                [APP_CFG_STARTUP_TASK_STK_SIZE-1], 5);
    OSTaskCreate(TASK_ShowGY86Data, (void*)0,
                (OS_STK*)&TASK_ShowGY86Datastk
                [APP_CFG_STARTUP_TASK_STK_SIZE-1], 6);

    OSStart();
    return 0;
}
```

接下来是各个任务的具体实现

代码 1-22 TASK_ChangeMotor 函数

```
//优先级为 5 的电机驱动任务
void TASK_ChangeMotor(void *p_arg){
    //创建信号量，当中断发生后，该任务才能执行
    PPM_Sem = OSSemCreate(0);
    while(1){
        INT8U err;
        //请求获取信号量
        OSSemPend(PPM_Sem, 0, &err);
        //获得通道一的值，即电机转速值
        uint16_t throttle = Rc_Data[1];
        if(err==OS_ERR_NONE){
            PWM_SetCompareAll(throttle);
        }
    }
}
```

代码 1-23 TASK_ShowGY86Data 函数

```

void TASK_ShowGY86Data(void *p_arg){
    while(1){
        GY86_GetData(&x, &y, &z, &AX, &AY, &AZ, &GX, &GY, &GZ);
        BLE_Printf ("01 %5d\n",AX);
        BLE_Printf ("02 %5d\n",AY);
        BLE_Printf ("03 %5d\n",AZ);
        BLE_Printf ("04 %5d\n",GX);
        BLE_Printf ("05 %5d\n",GY);
        BLE_Printf ("06 %5d\n",GZ);
    }
}

```

我们还需要编写一个外部中断的中断服务程序，用于获取遥控器的值，外部中断源为 TIM2 时钟。与上学期编写的 TIM2_IRQHandler 不同点在于，加入 OS 后，我们需要考虑对临界资源的保护以及出中断的进一步调度问题，为此，OS 已经给我写好了系统调用函数 OSIntEnter()和 OSIntExit()，分别用于进出中断程序。

代码 1-24 TIM2_IRQHandler 中断服务程序

```

void TIM2_IRQHandler(void){
    OS_CPU_SR  cpu_sr = 0u;
    OSIntEnter();
    if (TIM_GetITStatus(TIM2, TIM_IT_CC1) != RESET){
        uint32_t Rc_Val = TIM_GetCapture1(TIM2);
        OS_ENTER_CRITICAL();
        if(Rc_Val > 2050){
            Cal = 0;
        }
        else if(Cal < 8){
            Rc_Data[Cal] = Rc_Val;
            Cal++;
            if(Cal == 8){
                OSSemPost(PPM_Sem);
                Cal=0;
            }
        }
        OS_EXIT_CRITICAL();
        TIM_ClearITPendingBit(TIM2, TIM_IT_CC1);
    }
    OSIntExit();
}

```

1.3.5 SystemView 模块移植和使用

1.3.5.1 SystemView 移植

首先，在项目的工程目录中创建一个名为 SystemView 的新文件夹，用来存放与 SystemView 相关的例程文件。接下来，将 SystemView 官方提供的代码文件中，包含 Config、Sample 和 SEGGER 文件夹里的相关文件，复制并添加到刚刚创建的 SystemView 文件夹中。移植成功后，Keil 中导入文件如下图所示，其中红杠部分是官方例程中不建议导入的文件。

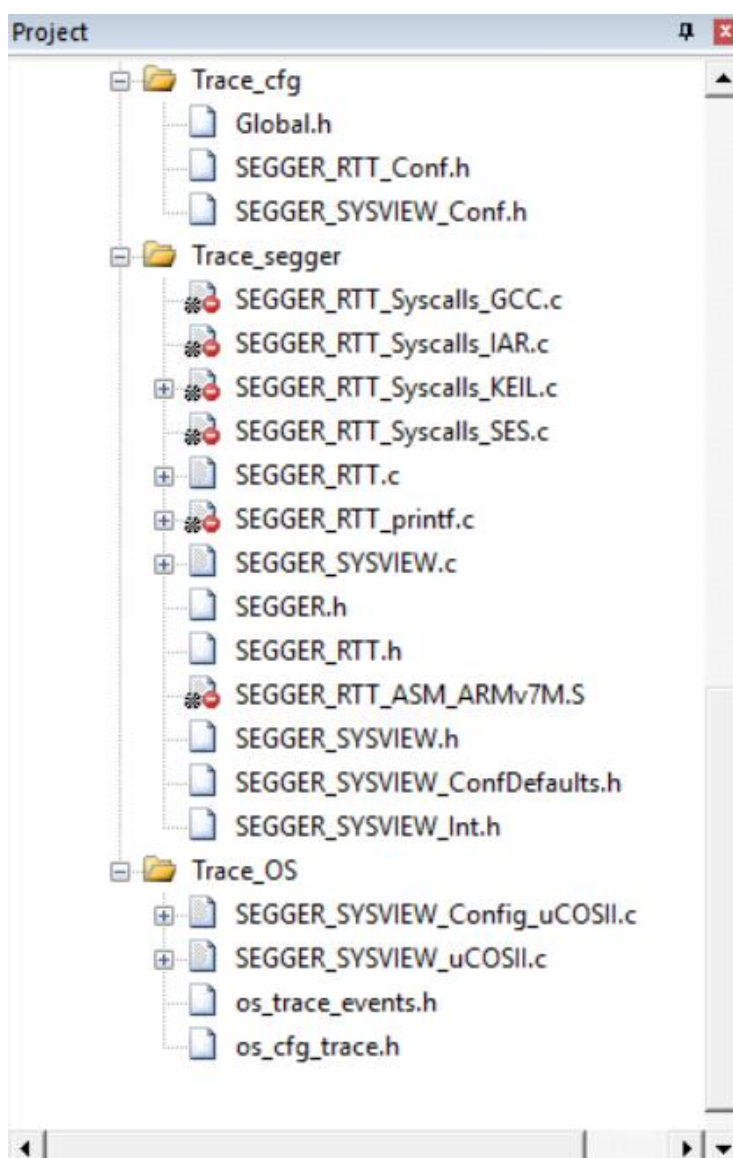


图 1-26 SystemView 的移植目录

1.3.5.2 SystemView 参数配置

为了使 SystemView 的程序能够正常监控操作系统的运行，我们需要将操作系统的运行参数，如系统时钟、中断函数名称等，写入 SystemView 的程序段中，因此才可以使 SystemView 与操作系统同步运行，进行操作系统的监控工作。

我们首先需要配置 os_cfg.h 文件，在 os_cfg.h 中启用 μ C/OS-II 操作系统中的跟踪记录器功能，跟踪 API 的进入和退出事件，如以下代码所示。

代码 2-1 os_cfg.h 配置

```
#define OS_TRACE_EN 1u
// 启用 (1) 或禁用 (0)  $\mu$ C/OS-II 跟踪工具
#define OS_TRACE_API_ENTER_EN 1u
// 启用 (1) 或禁用 (0)  $\mu$ C/OS-II 跟踪 API 进入工具
#define OS_TRACE_API_EXIT_EN 1u
// 启用 (1) 或禁用 (0)  $\mu$ C/OS-II 跟踪 API 退出工具
```

其次我们需要配置 SystemView 宏定义，为其提供必要的时间戳和中断号信息，通过直接读取特定内存地址（如 0xE0001004 和 0xE000ED04）来获取系统的时间和中断状态，以便对实时数据进行准确的分析和记录。

代码 2-2 SEGGER_SYSVIEW_Conf.h 配置

```
// 事件时间戳
#define SEGGER_SYSVIEW_GET_TIMESTAMP() (*(U32 *) (0xE0001004))
#define SEGGER_SYSVIEW_TIMESTAMP_BITS 28
// 中断 ID
#define SEGGER_SYSVIEW_ID_BASE 0x20000000
#define SEGGER_SYSVIEW_ID_SHIFT 0
#define SEGGER_SYSVIEW_GET_INTERRUPT_ID() ((*(U32 *) (0xE000ED04)) & 0x1FF)
// 解锁操作
#define SEGGER_SYSVIEW_UNLOCK() SEGGER_RTT_UNLOCK()
```

然后我们着重需要配置 SEGGER_SYSVIEW_Config_uCOSII.c 文件，这个文件是 SystemView 与操作系统强相关的文件，我们要在其中配置操作系统的名称、版本、时钟等信息。

代码 2-3 SEGGER_SYSVIEW_Conf_uCOSII.c 配置

```
// The application name to be displayed in SystemViewer
#define SYSVIEW_APP_NAME "ARM Cortex-M Demo"
// The target device name
#define SYSVIEW_DEVICE_NAME "Cortex-M Device"
```

```
// The lowest RAM address used for IDs (pointers)
#define SYSVIEW_RAM_BASE (0x20000000)
// Frequency of the timestamp.
// Must match SEGGER_SYSVIEW_GET_TIMESTAMP in SEGGER_SYSVIEW_Conf.h
#define SYSVIEW_TIMESTAMP_FREQ (84000000u)
// System Frequency.
// SystemcoreClock is used in most CMSIS compatible projects.
#define SYSVIEW_CPU_FREQ (84000000u)
```

1.3.5.3 工程文件中添加 SystemView 接口

完成前面的文件移植和参数配置后，我们需要将 SystemView 中的初始化函数、追踪函数添加进操作系统的运行代码中。操作系统通过运行追踪函数，即可将操作系统运行状态、任务状态等信息传递给 SystemView，我们即可通过 SystemView 获知操作系统运行的信息。

首先需要在主函数 main() 所在文件引入 SEGGER_SYSVIEW.h 的头文件。其次我们需要在主函数 main() 中的 OSInit() 后，加入 SystemView 的初始化函数 SEGGER_SYSVIEW_Conf()，进行必要的启动工作。随后加入 SEGGER_SYSVIEW_Start() 函数，即可启动 SystemView，进行任务追踪的任务。注意这里 SystemView 初始化代码和启动代码需要放在 OS 初始化代码的前面。添加部分代码如下。

代码 2-4 将 SystemView 接入操作系统

```
#include "SEGGER_SYSVIEW.h"
int main(void) {
    SEGGER_SYSVIEW_Conf(); //SystemView 初始化代码
    SEGGER_SYSVIEW_Start(); //SystemView 启动代码
    OSInit(); //初始化μC/OS

    //创建操作系统任务
    OSStart();
    return 0;
}
```

1.3.5.4 应用 SystemView 进行系统监测

配置完成上述步骤之后，即可进行操作系统的运行监控。此处，我们可通过 SEGGER 公司开发的配套“SEGGER SystemView”软件进行操作系统的监控。在这个软件中，提供了完善的可视化工具帮助我们获知操作系统的运行状态，并且

可以监控系统中每个任务的名称、优先级等基本的信息，以及运行的次数、时间等动态信息。

首先，需要将 Stm32 开发板通过 JLink 与电脑进行连接。因为我们使用的开发板的烧录工具是板载 STLink，因此需要在 STM32 官网下载工具软件，将 STLink 转换成 JLink 之后继续使用。通过 JLink 将程序烧录进开发板后，打开 SEGGER SystemView 软件，点击“Target”板块中的“Start Recording”即可开始进行操作系统的监控。

通过“System”界面，我们可以查看操作系统的概览信息，包括操作系统的名称、CPU 的时钟频率等基本信息，同时也能查看操作系统首次上电后，持续运行的时间、发生的事件次数等信息。除此之外，对于任务的概览信息也可以在此处查看。此界面概览见下图所示：

System	
Property	Detail
Cycle Period	11.904 ns
Time Offset	-01:26.503 848
▼ Recording	
Title	
Author	
Description	
Host Time	
Duration	01:30.343 940
Number of Events	13 561
> Event Frequency	418/s
Overflows	1 (5 869 Events, 12.607 s lost)
▼ Analysis	
Analysis Duration	02:15.16
Record Size	61.738 kB
> Throughput	1.89 kB/s
> Parse Frequency	381/s
▼ Tasks	
Number of Tasks	6
Task Switches	3 212
Task Time	01:17.676 987

图 1-27 SystemView 界面概览

通过进程主界面选择“Contexts”界面，我们可以查看操作系统中各个任务、进程的具体信息，包括名称、栈空间等静态信息，以及运行次数、总运行时间等动态信息。我们可以通过其与左上角任务运行板块的结合，以此了解到每一个任务发生的具体时间，与发生的次数，从而让我们对操作系统中的任务进程进行大致的观测。此界面概览见下图所示：

第二章 系统测试

2.1 μ C/OS-II 移植测试

2.1.1 测试目的

检验 μ C/OS II 移植是否成功实现，是否真正实现了多任务并发执行以及任务之间的切换。

2.1.2 测试步骤

- 选用板载 LED 灯（PA5 引脚），创建文件 LED.c,初始化相应寄存器，实现点灯和灭灯函数；在 Task.c 中创建两个任务；再在 main.c 中创建任务，交由 OS 调度，查看是否实现交替闪烁。
- 烧录项目代码后再次打开 SystemView 软件，观察两个任务是否正确切换。

2.1.3 测试代码

通过配置对应寄存器，可以实现板载 LED 灯的点亮和熄灭，由此我们定义两个任务 TASK1 和 TASK2 分别控制亮灯和熄灯,通过 OSTimeDly 实现任务切换，在主函数中分别创建两个任务，设定优先级为 5 和 6，即复位后灯亮。

代码 2-1 移植测试任务

```
void TASK1(void *p_arg){
    while(1){
        LED_ON();
        OSTimeDly(1000);
    }
}
void TASK2(void *p_arg){
    while(1){
        LED_OFF();
        OSTimeDly(1500);
    }
}
```

```
//main.c
OSTaskCreate(TASK1, (void*)0, (OS_STK*)&TASK1stk[APP_CFG_STARTUP_TASK_
STK_SIZE-1], 5);
OSTaskCreate(TASK2, (void*)0, (OS_STK*)&TASK2stk[APP_CFG_STARTUP_TASK_
STK_SIZE-1], 6);
OSStart();
```

2.1.4 预期结果

板载 LED 灯在复位后变亮，之后有规律地闪烁，SystemView 软件监测界面两个任务正常交替运行。

2.1.5 测试结果

烧录代码后复位执行，板载 LED 灯在复位后立即变亮，之后有定时闪烁，SystemView 软件监测界面两个任务正常交替运行。Event List 窗口展示任务切换如下所示。

图 2-1 Event List 窗口任务切换情况

28	0.000 000 000	my_task_1_t	Task Run	Runs for 0.000 us
29	0.000 000 000	my_task_1_t	Task Block	Pending
30	0.000 000 000	my_task_1_t	Task Run	Runs for 0.000 us
31	0.000 000 000	my_task_1_t	Task Block	my_task_0_t, Pending
32	0.000 000 000	my_task_0_t	Task Run	Runs for 0.000 us
33	0.000 000 000	my_task_0_t	Task Block	uC/OS-II Stat, Pending

2.2 项目集成测试

2.2.1 测试目的

验证整体功能是否正常，确保各个系统模块加上 $\mu\text{C}/\text{OS II}$ 之后能够协同工作，实现多任务下及时的电机控制与 GY86 数据反馈。

2.2.2 测试步骤

- 裸板编写集成测试代码过后，加入 SystemView 接口函数，烧录后打开 SystemView 监测软件，检查任务是否正确交替执行；
- 在四轴飞行器的硬件连接基础上，打开遥控器并使用遥控器控制油门摇杆，通过接收机接收信号。检查接收机接收信号后，是否正确传递至电调并控制四个

电机转速；检查蓝牙模块是否能够将数据传输到蓝牙接收端。

2.2.3 预期结果

裸板下测试时 SystemView 正确显示数据传输和电机控制两个任务地切换，TIM2 中断（ISR44）正确触发；装载到飞行器后各模块工作均能完成各自任务，并实现正确的数据传输。

2.2.4 测试结果

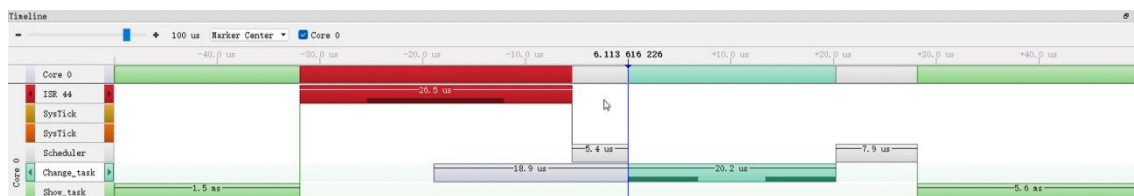
裸板下测试时 SystemView 的事件窗口如下图所示，可以看到名称为 "Change_task" 的电机控制任务、名称为 "Show_task" 的数据传输任务以及 ISR44 中断服务程序实现正确切换。

图 2-2 SystemView 事件窗口

#	Time	Context	Event	Resource	Detail
55571	38.505 695 357	Change_task	Task Run		Runs for 20.250 us
55572	38.505 702 143	Change_task	OSSemPend		Returns OS_ERR_NONE after 16.995 ms
55573	38.505 708 786	Change_task	OSSemPend	* ? (2)	pevent=? (2)
55574	38.505 715 607	Change_task	Task Block		Pending
55575	38.505 723 571	Show_task	Task Run		Runs for 16.968 ms
55576	38.510 365 107	ISR 44	ISR Enter		Runs for 7.929 us
55577	38.510 373 036	ISR 44	ISR Exit		Returns to Show_task
55578	38.511 853 095	ISR 44	ISR Enter		Runs for 8.190 us
55579	38.511 861 286	ISR 44	ISR Exit		Returns to Show_task
55580	38.513 831 000	ISR 44	ISR Enter		Runs for 8.190 us
55581	38.513 839 190	ISR 44	ISR Exit		Returns to Show_task
55582	38.515 331 190	ISR 44	ISR Enter		Runs for 8.190 us
55583	38.515 339 381	ISR 44	ISR Exit		Returns to Show_task
55584	38.516 891 952	ISR 44	ISR Enter		Runs for 8.190 us
55585	38.516 900 143	ISR 44	ISR Exit		Returns to Show_task
55586	38.517 892 333	ISR 44	ISR Enter		Runs for 8.190 us
55587	38.517 900 524	ISR 44	ISR Exit		Returns to Show_task
55588	38.518 892 714	ISR 44	ISR Enter		Runs for 8.190 us

切换到 Timeline 时间线窗口，如下图所示，电机控制任务、数据传输任务以及 ISR44 中断服务程序能够正确实现 OS 调度下的 CPU 使用权切换。

图 2-3 SystemView 时间窗口



装载到飞行器后，油门摇杆能够正确控制信号的发送，接收机正常接收并传递信号给电调，电调成功控制四个电机的转速，蓝牙模块能够成功收集并传输电机转速和飞行数据至上位机中，上位机蓝牙调试助手显示数据如下图所示。

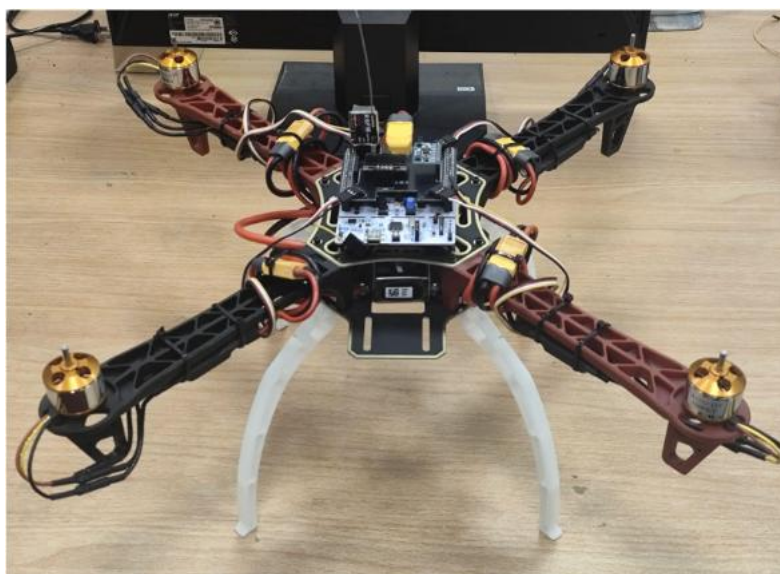


图 2-4 蓝牙调试助手显示数据



图 2-5 蓝牙调试助手显示数据

结合以上现象，集成了 $\mu\text{C}/\text{OS II}$ 的飞行器项目能够正确实现多任务并发下的电机控制和数据传输逻辑。

第三章 知识技能学习情况

3.1 互斥与临界区

3.1.1 互斥与临界区概念

互斥:我们把一个时间段内只允许一个任务使用的资源称为临界资源。对临界资源访问的这种排他性就是互斥。

临界区:每个任务访问共享资源的那段程序称为临界区 (Critical Section)。在临界区不允许任务切换,如果在访问共享资源时进行任务切换,可能发生灾难性后果。因此,在进入临界区访问共享资源前,采用关中断,给调度器上锁或使用信号量的方法达到互斥的目的。

信号量: $\mu\text{C}/\text{OS II}$ 采用信号量作为任务与任务间的同步和互斥机制。信号量 (Semaphore), 是在多线程环境下使用的一种设施, 是可以用来保证两个或多个关键代码段不被并发调用。在进入一个关键代码段之前,线程必须获取一个信号量;一旦该关键代码段完成了,那么该线程必须释放信号量。其它想进入该关键代码段的线程必须等待直到第一个线程释放信号量。

$\mu\text{C}/\text{OS II}$ 中的信号量由两部分组成:一部分是信号量的计数值(16 位无符号整数), 另一部分是等待该信号量的任务组成的任务等待队列。

$\mu\text{C}/\text{OS II}$ 提供了三种信号量:

- 用于任务同步的信号量。
- 任务对共享资源互斥访问的互斥信号量。
- 对系统多个资源管理的信号量。

3.1.2 三种进入临界区的方式

$\mu\text{C}/\text{OS II}$ 提供了三种进入临界区的方式。

第一种是进入临界区时直接调用一个指令来禁用所有中断,退出时则直接使能中断。代码如下:

代码 3-1 第一种临界区进入方式

```

#if OS_CRITICAL_METHOD == 1
#define OS_ENTER_CRITICAL() __asm volatile ("cpsid i" : : : "memory")
#define OS_EXIT_CRITICAL() __asm volatile ("cpsie i" : : : "memory")
#endif

```

这种方式虽然简单易行，但是无法满足嵌套的情况。如果有两层临界区保护，在退出内层临界区时就会开中断，使外层的临界区也失去保护。同时该方式可能导致系统延迟变大。'cpsid' 直接关闭所有中断，会导致高优先级中断（如时钟中断）无法执行，从而影响系统实时性。

第二种是在关中断前保存之前的标志寄存器内容到栈中(将中断状态保存在栈中)，退出临界区的时候从栈中恢复之前保存的状态。

代码 3-2 第二种临界区进入方式

```

#if OS_CRITICAL_METHOD == 2
__asm void OS_ENTER_CRITICAL(void) {
    MRS R0, PRIMASK ;; 将 PRIMASK 的内容加载到 R0
    CPSID I ;; 禁用中断 (设置 PRIMASK = 1)
    STMDB SP!, {R0} ;; 保存原始 PRIMASK 到堆栈
    BX LR ;; 返回
}
__asm void OS_EXIT_CRITICAL(void) {
    LDMIA SP!, {R0} ;; 从堆栈中恢复原始 PRIMASK
    MSR PRIMASK, R0 ;; 恢复 PRIMASK 寄存器
    BX LR
}
#endif

```

一些编译器不能很好地优化内联代码，无法知道堆栈指针已经被更改，此时再想去访问堆栈中的数据极大可能出现错误的值，从而导致应用程序的崩溃。比如：在函数调用函数，参数过多，使用栈传递参数时，便会出现问题。

而第三种方式是目前来说使用比较多的一种，在关中断前，使用局部变量 `cpu_sr` 保存中断状态。代码如下所示：

代码 3-3 第三种临界区进入方式

```

; 保存中断状态并关中断
OS_CPU_SR_Save PROC
    MRS R0, PRIMASK ; 读取 PRIMASK 寄存器

```



```
CPSID I ; 关闭中断
BX LR ; 返回
ENDP

; 恢复中断状态
OS_CPU_SR_Restore PROC
    MSR PRIMASK, R0 ; 恢复 PRIMASK 寄存器
    BX LR ; 返回
ENDP
```

这种方式支持中断多层嵌套，也没有方法二中的编译器优化问题，在不同平台中广泛使用。

3.2 Makefile 与 Cmake 工程构建

3.2.1 程序编译过程

1) **预处理**：预处理器读取源代码，并根据预处理器指令（例如，"#include"、"#define"、"#ifdef"等）进行头文件替换、宏扩展、条件编译和删除注释。

2) **编译**：把预处理完的文件进行一系列词法分析、语法分析、语义分析及优化后被编译器 cc1 编译成相应的汇编代码文件(.s)，然后将汇编代码转变为机器可执行的代码也就是目标文件(.o)。

3) **链接**：链接器将来自编译阶段的多个目标文件与库组装在一起，解决函数与变量的引用问题和目标文件之间的依赖问题，并生成可执行文件。

3.2.2 交叉编译工具链

ARM-GCC 是一个工具链，为基于 ARM 的处理器提供 GNU 编译器集合(GCC)。它是一个开源编译器、链接器、汇编器和其他开发工具的集合，可以为 ARM 架构进行软件开发。

它支持对各种 ARM 微控制器和处理器（包括 Cortex-M、Cortex-R 和 Cortex-A 系列）的 C、C++ 和汇编语言代码进行编译。它提供了优化的代码生成、调试支持以及大量的功能和优化，以增强基于 ARM 的嵌入式系统的开发过程。

ARM-GCC 工具链的关键组件包括：

1) GCC 编译器: GNU 编译器集合提供了前端编译器, 支持 ARM 架构的 C、C++ 和汇编语言。它将源代码翻译成特定于目标 ARM 处理器的机器代码。

2) Binutils: Binutils 软件包包括基本工具, 如汇编器、链接器和对象文件实用程序, 用于创建、操作和管理二进制文件和对象代码。

3) GDB 调试器: GNU 调试器 (GDB) 是一个强大的命令行调试器, 允许开发人员调试基于 ARM 系统上运行的程序。它支持断点、观察点、步入代码、检查变量等功能。

4) 库: ARM-GCC 包括一组标准库, 如 C 标准库 (libc)、C++ 标准库 (libstdc++) 和其他运行时库, 以便为嵌入式软件开发提供常用功能和语言支持。

5) 构建系统集成: ARM-GCC 可以集成到流行的构建系统中, 如 Make、CMake 和其他系统, 以实现构建过程的自动化和管理依赖性。

3.2.3 Makefile 基础

1) 定义变量

为了简化和自动化编译链接过程, 需要定义一些变量。这些变量包括:

- 生成中间文件存放位置: 用于存放编译过程中生成的中间文件。
- 是否 DEBUG: 决定是否生成调试信息。
- 优化等级: 设置编译器的优化级别。

需要指定的文件和选项包括:

- 源文件: 参与编译的所有 .c 和 .s 文件。
- 头文件: 参与编译的所有 .h 文件。
- 目标文件: 需要生成的所有 .o 文件。
- 编译选项:

① CFLAGS: 用于编译 C 文件的选项。

② ASFLAGS: 用于汇编文件的选项。

- 链接选项: LDFLAGS, 用于链接生成最终可执行文件的选项。

2) 编译链接依赖关系

最终需要生成三个文件：.elf、.hex 和 .bin。生成过程如下：

- 生成 .elf 文件：通过链接 .o 文件生成 .elf 文件。
- 生成 .hex 和 .bin 文件：通过 .elf 文件生成 .hex 和 .bin 文件。
- 生成 .o 文件：
 - ① 通过编译 .c 和 .h 文件生成部分 .o 文件。
 - ② 通过汇编 .s 文件生成部分 .o 文件。

3.2.4 Cmake 工程构建

CMake 是一个开源的跨平台自动化建构系统，它的优点是并不依赖于某特定编译器，并且支持自动生成 MakeFile。因此，我们选择使用 CMake 构建工程，增加系统的可移植性。使用 CMake 工具，需要在工程根目录编写 CMakeLists.txt 文件，在其中我们需要配置整个系统的构建信息，包括操作系统信息、处理器信息，以及源代码路径、编译和链接的命令。

由此，我们可以发现通过 CMake 配置整个工程构建的简便与强大之处。CmakeLists.txt 文件如下。

代码 3-4 CmakeLists.txt 文件

```
cmake_minimum_required (VERSION 3.15)
project (freely VERSION 1.0)

enable_language(C ASM)
set(CMAKE_C_STANDARD 99)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
set(EXECUTABLE ${PROJECT_NAME}.out)

aux_source_directory(. DIR_SRCS)
add_executable(${EXECUTABLE} ${DIR_SRCS})

target_compile_definitions(${EXECUTABLE} PRIVATE
    -DSTM32F401xx
)

target_include_directories(${EXECUTABLE} PRIVATE
```

```

    Include
    Core
)

target_compile_options(${EXECUTABLE} PRIVATE
    -mcpu=cortex-m4
    -mthumb
    -mfpv4-sp-d16
    -mfloat-abi=hard
    -fdata-sections
    -ffunction-sections
    -Wall
    $<$<CONFIG:Debug>:-Og>
)

target_link_options(${EXECUTABLE} PRIVATE
    -T${CMAKE_SOURCE_DIR}/STM32F401RETx_FLASH.ld
    -mcpu=cortex-m4
    -mthumb
    -mfpv4-sp-d16
    -mfloat-abi=hard
    # -specs=nano.specs
    -lc
    -lm
    -lnosys
    -Wl,-Map=${PROJECT_NAME}.map,--cref
    -Wl,--gc-sections
)

# Print executable size
# add_custom_command(TARGET ${EXECUTABLE}
#     POST_BUILD
#     COMMAND arm-none-eabi-size ${EXECUTABLE}
# )
# Create hex file
add_custom_command(TARGET ${EXECUTABLE}
    POST_BUILD
    COMMAND arm-none-eabi-objcopy -O ihex ${EXECUTABLE}
    ${PROJECT_NAME}.hex
    COMMAND arm-none-eabi-objcopy -O binary ${EXECUTABLE}
    ${PROJECT_NAME}.bin)

```

参考文献

- [1] 廖勇, 杨霞. 嵌入式操作系统[M]. 人民邮电出版社. 2017.
- [2] 汤小丹, 梁红兵, 哲凤屏等. 操作系统. 西安电子科技大学出版社[M]. 2018.
- [3] Joseph Yiu. The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors[M]. Elsevier Inc. 2014.
- [4] Jean J. Labrosse. μ C/OS-II User Manual. Micrium Press. 2015.
- [5] SEGGER. SystemView User Guide[M]. SEGGER Microcontroller GmbH. 2023.
- [6] STM32F4xx 中文参考手册[J]. 意法半导体
- [7] STM32 固件库使用手册[J]. 意法半导体
- [8] STM32F4 开发指南-寄存器版本[J]. 正点原子
- [9] 零死角玩转 STM32F401[J]. 野火

致谢

本报告的工作是在我们敬爱的指导教师廖勇老师的悉心指导下完成的。廖老师不仅以其渊博的专业知识为我们解答问题，还主动帮助我们分析问题，组织大家共同讨论并制定任务，让我们相互监督、共同进步。每当遇到难以解决的问题时，廖老师总是耐心地给予讲解，确保我们能够理解并解决。他从知识讲授到文档撰写，再到答辩技巧，都给予了无微不至的指导。正是廖老师的指导，我们才能明确每一步的具体要做的事情，将看似不可能完成的任务一步步变成现实。对于廖老师的悉心指导，我们衷心表达感谢和敬意！

此外，我们也要感谢学院给予我们独立完成课程设计的机会。在这个过程中，学院为我们提供了教师资源和学习资源的帮助，使我们能够将课堂中学到的知识应用于实际的计算机系统结构能力。这不仅增强了我们的实践操作水平，还培养了我们的合作交流与分工能力。

我们还要感谢在课程设计完成的过程中帮助过我们的老师、前辈和同学们。感谢你们的提出的宝贵经验和建议，让我们的工作少走弯路，没有你们的耐心指导，就没有项目的顺利完成。感谢每一位进行知识分享的同学，他们的独特思想和创造力让我们受益匪浅。同时，感谢同组成员之间的齐心协力，我们共同克服困难，顺利地完成了移植 $\mu C/OS-II$ 到 STM32 芯片上，并基于 $\mu C/OS-II$ 运行四轴飞行器的艰巨任务！

最后，我们再次由衷地感谢廖勇老师和所有帮助过我们的人。廖老师的专业知识、严谨的治学精神和教学激情让我们深受感染。感谢每一位帮助过我们的人，你们的支持和鼓励是我们成功的重要动力。



信息与软件工程学院

挑战性项目 II 中期报告

课程名称： 进阶式挑战性综合项目 II（嵌入式系统）

课题名称： 四轴飞行器设计

指导教师： 廖勇

所在系别： 软件工程（嵌入式系统）

执行学期： 2024-2025 学年 2 学期

学生信息：

序号	学号	姓名
1（组长）	2023090903028	曾欣晨
2	2022130102013	成棋伟
3	2023090903010	吴欣锐
4	2023090901015	冉昕堃

目录

第一章 项目分析与总体设计	1
1.1 阶段性目标分析	- 1 -
1.2 总体设计方案	1
第二章 针对复杂工程问题的分析与实现	4
2.1 用 KEIL 重构 uC/OS II 新工程	4
2.1.1 μ C/OS II 操作系统架构	4
2.1.2 工程重构实现	6
2.2 ARM 内核事件驱动	11
2.2.1 事件驱动原理	11
2.2.2 任务堆栈初始化	14
2.3 ARM 内核时间驱动	16
2.3.1 时间驱动原理	16
2.3.2 时间驱动实现	16
2.4 uC/OS II 启动代码设计	22
第三章 前期完成度和后续实施计划	28
参考文献	29

第一章 项目分析与总体设计

1.1 阶段性目标分析

此次项目，我们小组的设计思路分为：硬件模块功能，操作系统移植统筹，电机控制算法。在上学期的项目中，我们已实现了硬件等响应模块的功能，在开启本学期的任务之前，我们先回顾一下第一部分的成果：

- 四轴飞行器机架搭建
- PCB 转接板的设计，出板与焊接
- GY-86 的数据获取与分析
- 捕获遥控器接收机产生的 PWM 波信号
- 通过 STM32 对应引脚输出 PWM 波信号调节舵机转速

在上一学期的设计中，我们使用了循环来完成类似于操作系统的轮询操作，但这种方式会导致 CPU 因长时间等待信号被占用，并不能满足本挑战项目最后所要达到的实时性能，也不方便我们下学期添加新的姿态解算和 PID 反馈控制等任务。

为了使飞行器系统能够根据任务的重要性的时间紧迫性有逻辑，有次序的稳定运行，我们需要将整体架构重构，完成实时操作系统的移植。于是在本进阶式挑战性综合项目 II 的课程中，基于上一学期挑战性课程的裸机基础之上，我们需要实现实时操作系统的移植。

为了便于调试，我们要引入任务级调试工具以观察每个任务的运行情况，找出问题并进行优化改进。

1.2 总体设计方案

操作系统是管理计算机硬件和软件资源的应用程序，它需要分配 CPU 的资源、统筹资源供需的先后顺序、控制输入输出设备、管理文件系统等基本事务。在四轴飞行器上，用户希望在同一时间内有多个任务并发执行，而 MCU 一次只能顺序执行其中的一项任务。为了使各个程序能运行且正确地运行，需要合理的分配

先后顺序，从而让每个任务都得到足够的资源去运行，保证飞行器的稳定运行。

考虑到 STM32 本身的内存限制，以及对于实时性性能的要求，我们选择了 $\mu\text{C}/\text{OS II}$ 这一个操作系统。它包含了任务调度、任务管理、时间管理、内存管理、任务间通信等多种功能，而且稳定性强，代码开源，生态良好、资料丰富，方便初学者上手移植实现。

选定操作系统后，我们还需要查看各个任务在这个操作系统上的运行情况，例如运行开始时间，运行时间，抢占情况等。本小组采用的是名为 Systemview 的操作系统监视分析工具，它能够以条形图的形式，将所有任务的运行时间呈现出来，且与当前系统兼容良好，极大提高了开发效率。

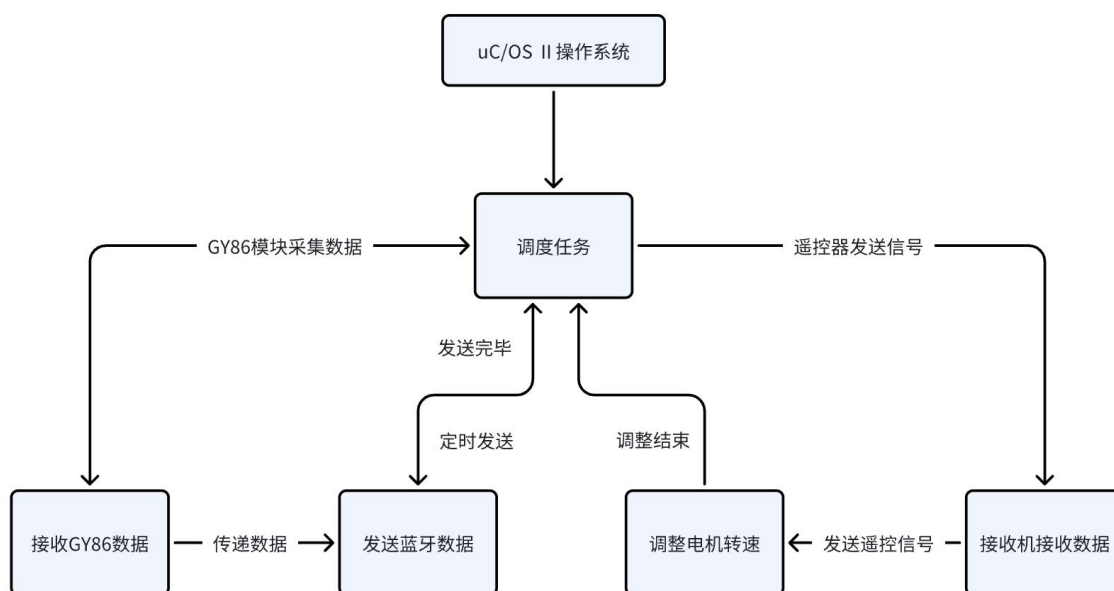


图 1-1 总体任务架构图

本次课程项目的总体设计如图 1-1 所示。为了使各个硬件模块之间能够形成良好的逻辑顺序，我们将四个硬件模块分为初始化和运行任务两个不同的函数。初始化用于在操作系统启动时就配置好，运行代码则是要等开始运行操作系统后才创建任务。所以我们有了如上的架构，将以往四个模块，分配成操作系统下的两个任务，一个任务负责改变电机转速，另一个任务负责读取 GY86 数据并通过蓝牙发送至上位机，这样的架构不仅方便从系统中移除，也方便以后有新的任务（如姿态解算）添加时，可以更加简单高效，不易出错。

为了实现 $\mu\text{C}/\text{OS}$ 下的多任务并发集成，我们综合考虑硬件模块初始化、数据传输需要以及电机控制逻辑，决定设计的系统业务逻辑如下：

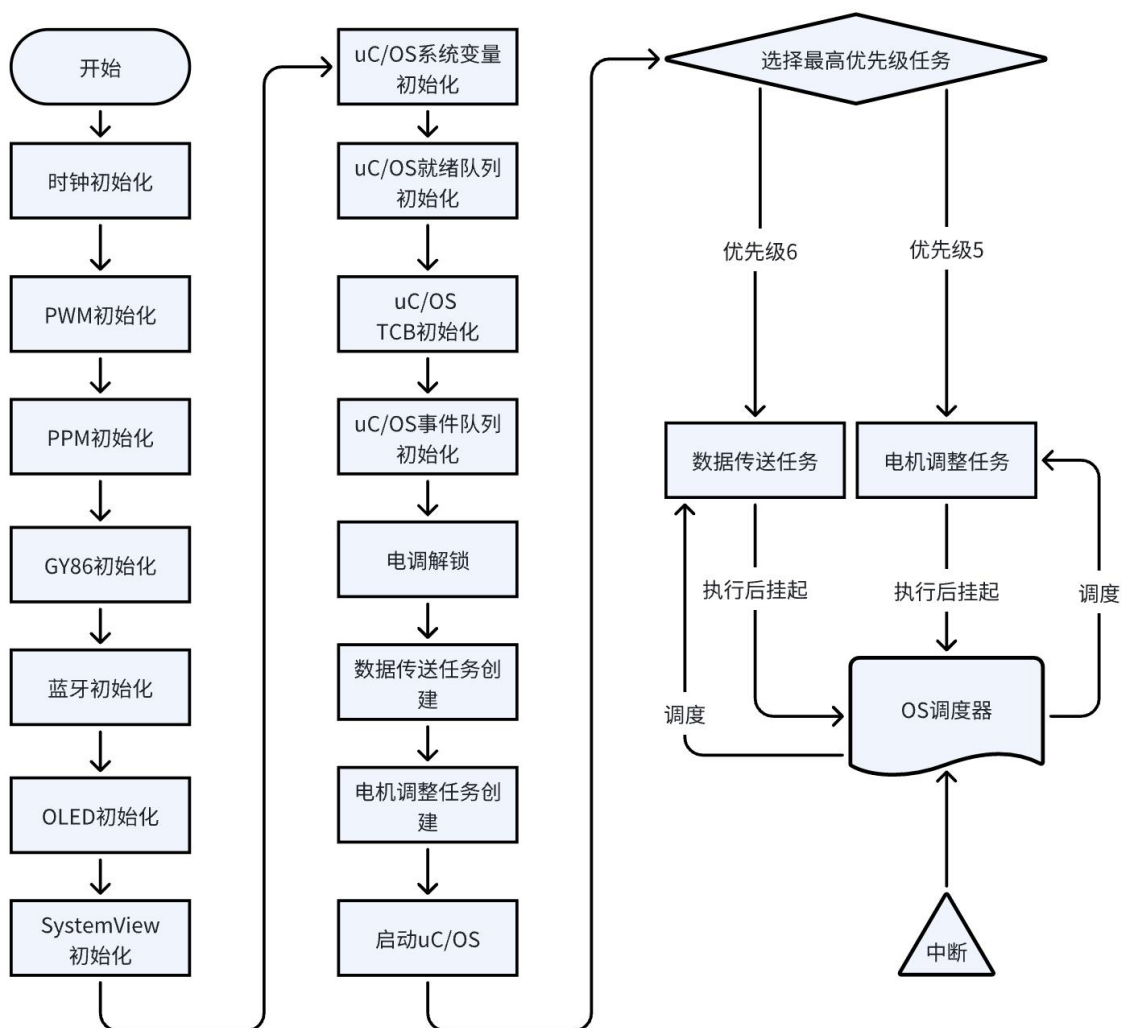


图 1-2 业务层逻辑流程图

第二章 针对复杂工程问题的分析与实现

2.1 用 KEIL 重构 $\mu\text{C}/\text{OS II}$ 新工程

2.1.1 $\mu\text{C}/\text{OS II}$ 操作系统架构

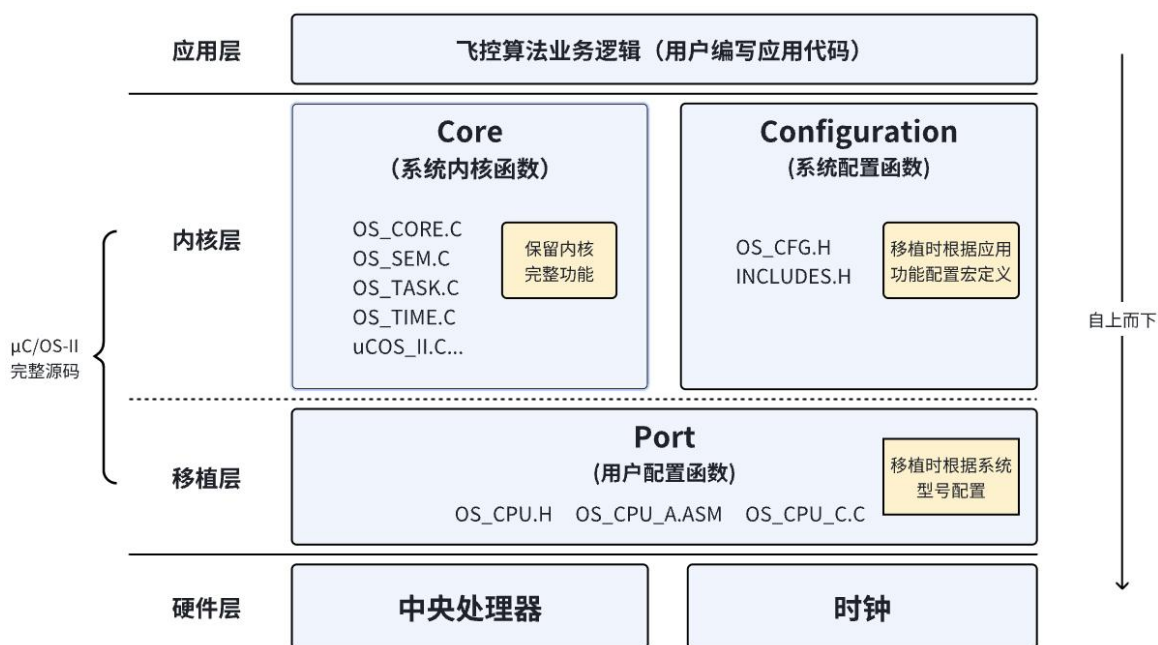


图 2-1 $\mu\text{C}/\text{OS II}$ 内核架构图

在利用 $\mu\text{C}/\text{OS II}$ 进行设计时，我们首先需要学习 $\mu\text{C}/\text{OS II}$ 内核结构、功能和函数，并且能够针对 STM32F401 开发板进行合理的设置与移植。以下是对 $\mu\text{C}/\text{OS II}$ 源代码内核的各个功能模块的详细介绍。

1) CORE（核心，一般不作修改）

① OS_CORE.c: $\mu\text{C}/\text{OS II}$ 内核的核心初始化文件，负责初始化任务调度器、创建 Idle 任务、事件控制块等。

② OS_TASK.c: 包含任务管理相关的函数，如任务的创建、删除、修改任务控制块（TCB）内容、任务延时等相关功能。

③ OS_FLAG.c: 用于实现标志（Flag）机制，服务于任务间的通信和同步。标志可以被设置、等待和清除。

④ OS_MEM.c: 实现与内存管理相关的函数, 可以动态分配和释放内存块, 并提供了内存池和固定大小内存块分配策略。

⑤ OS_TMR.c: 实现定时器功能, 允许用户设置定时器, 以在指定的时间点或时间间隔后触发相应的操作或任务。

⑥ OS_MUTEX.c: 用于实现互斥量 (Mutex) 机制, 提供实现互斥量的接口函数, 服务于任务间互斥访问共享资源。

⑦ OS_SEM.c: 用于实现信号量 (Semaphore) 机制, 提供实现信号量的接口函数, 服务于任务间的同步和互斥。

⑧ OS_MBOX.c: 用于实现邮箱 (Mailbox) 机制, 提供实现邮箱功能的接口函数, 服务于任务间的消息传递。

⑨ OS_Q.c: 用于实现队列 (Queue) 机制, 提供实现队列的接口函数, 服务于任务间通信。

⑩ uCOS_II.h: 包含 μ C/OS II 内核中各种数据结构的定义和函数声明, 如任务、事件、链表、信号量等。

2) PORT (根据处理器不同需要调整实现正确移植)

① OS_CPU_C.c: 包含与特定处理器相关的代码, 提供钩子函数、任务栈的初始化和系统时钟 Tick 的配置等。

② OS_CPU_A.asm: μ C/OS II 中汇编实现的上下文切换核心代码, 负责保存和恢复任务的上下文, 并通过调用 C 语言实现的函数完成任务切换操作。

3) CONFIG

① includes.h: 包含所有 μ C/OS II 的头文件和与处理器相关的头文件。

② os_cfg.h: μ C/OS II 提供了裁剪功能, 可以根据系统需求选择性地裁剪内核功能。这个文件用于配置这些功能的开启和关闭。

2.1.2 工程重构实现

首先, 我们需要在 keil 中添加 μ C/OS-II 源代码, 构建 os_cpu.h 等与处理相关代码文件, 并顺利通过编译。

1) 项目框架构建

新建一个文件夹 μ C/OS II, 并在文件夹下再新增三个文件夹 Core、Port 和 Cfg。

 os.h	2025/3/5 16:18	C Header 源文件	2 KB
 os_core.c	2025/4/8 16:19	C 源文件	87 KB
 os_dbg_r.c	2025/3/5 16:18	C 源文件	13 KB
 os_flag.c	2025/3/5 16:18	C 源文件	56 KB
 os_mbox.c	2025/3/5 16:18	C 源文件	32 KB
 os_mem.c	2025/3/5 16:18	C 源文件	20 KB
 os_mutex.c	2025/3/5 16:18	C 源文件	40 KB
 os_q.c	2025/3/5 16:18	C 源文件	43 KB
 os_sem.c	2025/4/8 18:35	C 源文件	30 KB
 os_task.c	2025/3/5 16:18	C 源文件	60 KB
 os_time.c	2025/3/5 16:18	C 源文件	11 KB
 os_tmr.c	2025/3/5 16:18	C 源文件	45 KB
 os_trace.h	2025/3/5 16:18	C Header 源文件	11 KB
 ucos_ii.c	2025/3/5 16:18	C 源文件	2 KB
 ucos_ii.h	2025/3/5 16:18	C Header 源文件	79 KB

图 2-2 μC/OS II 操作系统 Core 代码

CORE 文件夹包含了μC/OS II 内核的核心功能模块。这些模块提供了任务管理、时间管理、资源管理、任务通信和同步等核心功能，是 μC/OS II 内核的基础。

 os_cpu.h	2025/3/5 16:17	C Header 源文件	11 KB
 os_cpu_a.asm	2025/3/5 16:17	Assembler Source	17 KB
 os_cpu_c.c	2025/3/9 18:41	C 源文件	36 KB
 os_dbg.c	2025/3/5 16:17	C 源文件	14 KB

图 2-3 μC/OS II 操作系统 Ports 代码

Ports 文件夹中的文件包含了针对不同处理器平台的移植代码。每个处理器平台都有不同的硬件架构和操作系统接口，因此需要根据处理器的特点进行移植，以确保 μC/OS II 内核可以在特定处理器上正确地运行。

 os_cfg.h	2025/3/5 16:17	C Header 源文件	11 KB
--	----------------	--------------	-------

图 2-4 μC/OS II 操作系统 Cfg 代码

CONFIG 文件夹中的文件用于配置 μC/OS II 内核的各种功能和选项。CORE、PORT 和 CONFIG 是 μC/OS II 内核的三大模块，通过这些模块的工作协同，我们可以根据目标系统的需求定制和优化 μC/OS II 内核，使其在 STM32F401 开发板

上高效运行。

2) 项目导入

打开 keil 5, 把它们导入到项目中。选择 project -> new uversion project, 然后选好芯片型号: STM32F401RETx。

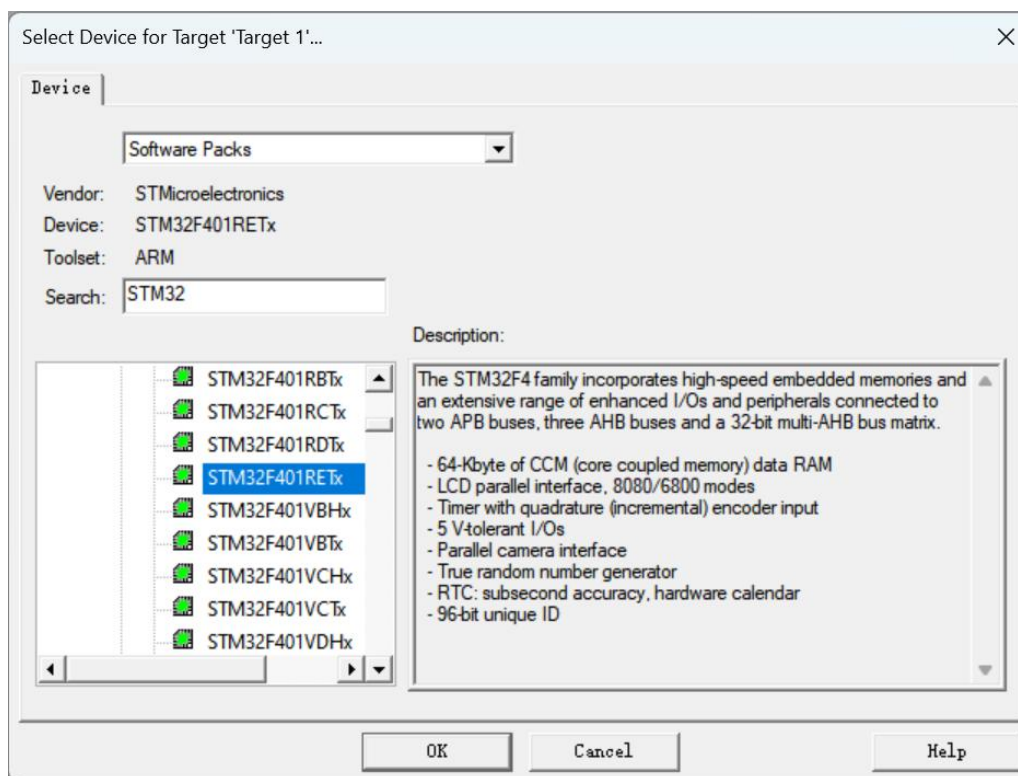


图 2-5 新建工程中芯片选择页面

3) 右键单击 target 1-add group 新建文件夹，并重命名为 Core、Port 和 Cfg，右键单击文件夹，add existing files to group ‘xxx’ 添加文件。

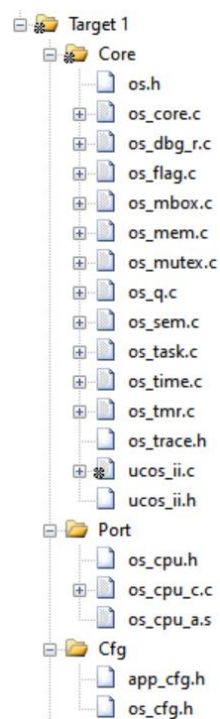


图 2-6 为 keil 中文件夹添加需要编译的文件

4) 添加头文件路径。打开魔术棒，在 C/C++和 Asm 里的 Include Path 中写入 C 程序和汇编文件的路径。

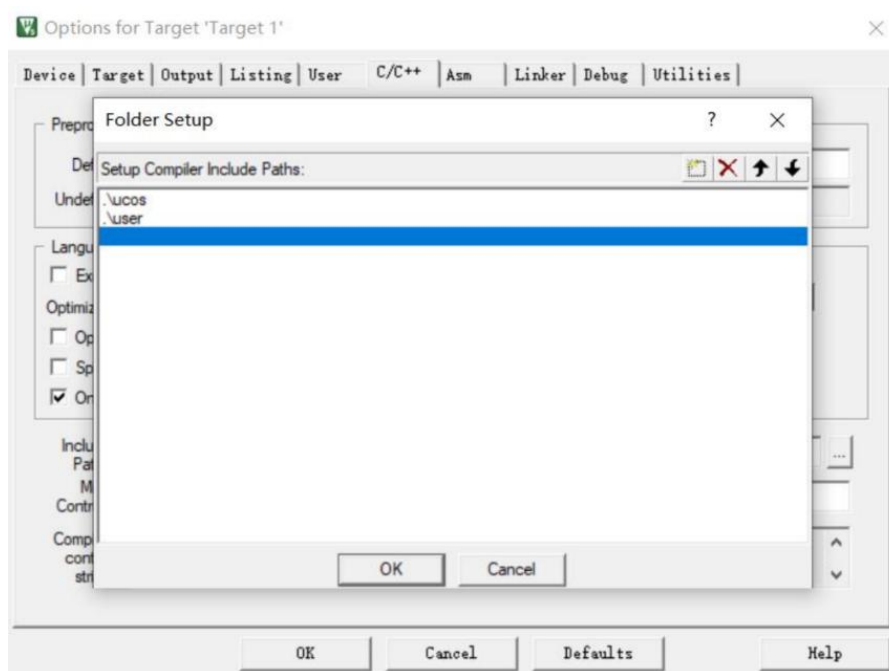


图 2-7 添加头文件所在路径

5) 解决若干编译报错

① 缺少文件或者找不到文件。

.\ucos\ucos_ii.h(45): error: #5: cannot open source input file "xxx": No such file or directory

解决方案：手动在 ucoss 中添加相应文件。

② INTxxU 型变量未定义

.\ucos\ucos_ii.h(1207): error: #20: identifier "INTxxU" is undefined

解决方案：在 os_cpu.h 中用 typedef 定义相应变量。32 位单片机上，32U 对应 unsigned int，16U 对应 unsigned short，8U 对应 unsigned char。

代码 2-1 main 函数编写

```
typedef unsigned char INT8U;
typedef unsigned int INT32U;
typedef unsigned short INT16U;
```

③ 由非 INTxxxU 型变量未定义引发的错误：

.\ucos\ucos_ii.h(719): error: #20: identifier "xxx" is undefined

解决方案：在 os_cfg.h 里，用#define 补上所需的宏定义

代码 2-2 增添宏定义相关参数值

```
#define OS_MAX_TASKS 64
#define OS_LOWEST_PRIO 64
#define OS_TASK_IDLE_STK_SIZE 64
```

④ 宏定义中的变量未定义，具体解决办法和错误类型 4 相同。

.\ucos\ucos_ii.h(1482): error: #35: #error directive: "OS_CFG.H, Missing OS_FLAG_EN: Enable (1) or Disable (0) code generation for Event Flags"

代码 2-3 在 os_cfg.h 中添加的宏定义

```
#define OS_FLAG_EN 0
```

⑤ 因为找不到指定函数导致链接错误：

.\Objects\baogao.axf: Error: L6218E: Undefined symbol

解决方案：查看报错中缺少的函数并添加。因为函数的声明和定义的语法规

则区别很大，编译器可以轻松地区分什么是声明，什么是定义。如果分别编译单个文件时，只有声明没有定义，编译器暂时不会报错，会尝试在链接时去其他文件中找相应的函数定义，但是如果链接时还没有找到函数体，编译器就会报错。新建一个 `os_cpu.c`，补充缺少的函数定义。

代码 2-4 `os_cpu.c` 中与处理器相关的代码

```
#include "os_cpu.h"
void SystemInit(){}
void OSInitHookBegin(){}
void OSInitHookEnd(){}
void OSTCBInitHook(){}
void OSTaskCreateHook(){}
void OSTaskIdleHook(){}
void OSTaskReturnHook(){}
OS_STK *OSTaskStkInit(void (*task)(void *p_arg),
                      void *p_arg, OS_STK
                      *ptos, INT16U opt){
    return ptos;
}
void OSTaskSwHook(){}
void OS_CPU_ExceptStkBase(){}
void OS_KA_BASEPRI_Boundary(){}

```

⑥ 临界区函数未定义错误：

.\Objects\baogao.axf: Error: L6218E: Undefined symbol OS_ENTER_CRITICAL (referred from ucos_ii.o).

.\Objects\baogao.axf: Error: L6218E: Undefined symbol OS_EXIT_CRITICAL (referred from ucos_ii.o).

warning: #223-D: function "OS_EXIT_CRITICAL" declared implicitly

解决方案：在 `os_cfg.h` 中加上临界区保护函数：

代码 2-5 `os_cfg.h` 中临界区保护相关代码

```
extern OS_CPU_SR;
//在 os_cfg.h 中定义为 unsigned char 类型。
extern OS_CPU_SR OS_CPU_SR_Save(void);
//引用汇编文件 os_cpu_a.asm 中的函数
extern void OS_CPU_SR_Restore(OS_CPU_SR);
//引用汇编文件 os_cpu_a.asm 中的函数

```

```
extern void OSCtxSw(void);
//引用汇编文件 os_cpu_a.asm 中的函数
#define OS_ENTER_CRITICAL() (cpu_sr = OS_CPU_SR_Save())
#define OS_EXIT_CRITICAL() (OS_CPU_SR_Restore(cpu_sr))
#define OS_CRITICAL_METHOD 3u
//表示采用第三种临界区保护方式
```

⑦ 编译文件并修改相关警告

.\ucos\os_core.c(723): warning: #223-D: function "OSIntCtxSw" declared implicitly

.\ucos\os_core.c(876): warning:#223-D: function "OSStartHighRdy" declared implicitly

解决方案：这里需要使用到 `extern` 关键字。这两个函数名标号在其他文件中定义，而在 `os_core.c` 中没有定义，故需使用 `extern` 关键字。

代码 2-6 `os_core.c` 中补充 `extern` 关键字部分代码

```
extern void OSIntCtxSw(void);
extern void OSStartHighRdy(void);
```

最终实现了 0 error 0 warning。

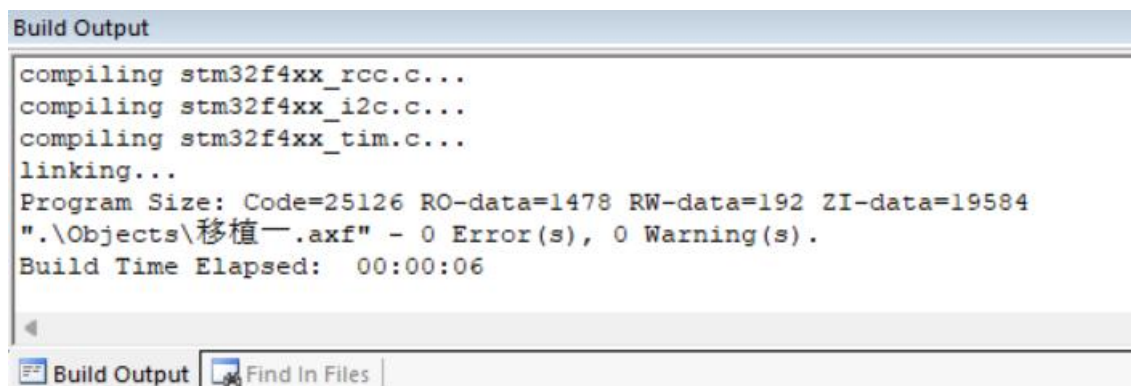


图 2-8 编译结果图

2.2 ARM 内核事件驱动

2.2.1 事件驱动原理

在 ARM 处理器中，事件驱动是指将可能发生的事情进行编号，通过这些编号，

我们可以设置硬件的跳转地址。当这些预设的事件发生时，由硬件发出信号，告知 MCU 发生了什么，再通过指令跳转到这事件预设的中断服务函数中完成对应的处理。

Cortex-M4 内核中跟中断相关的部分主要是 NVIC(嵌套向量中断控制器)管理模块，它负责控制来自内核内部的异常和来自外部的中断，最多能够支持 240 个 IRQ(中断请求)、1 个不可屏蔽中断(NMI)、1 个 SysTick(系统节拍)定时中断及多个系统异常。多数 IRQ 由定时器、I/O端口和通信接口(如 UART 和 IC)等外设产生。NMI 通常由看门狗定时器或掉电检测器等外设产生，其余的异常则是来自处理器内核，中断还可以利用软件生成。

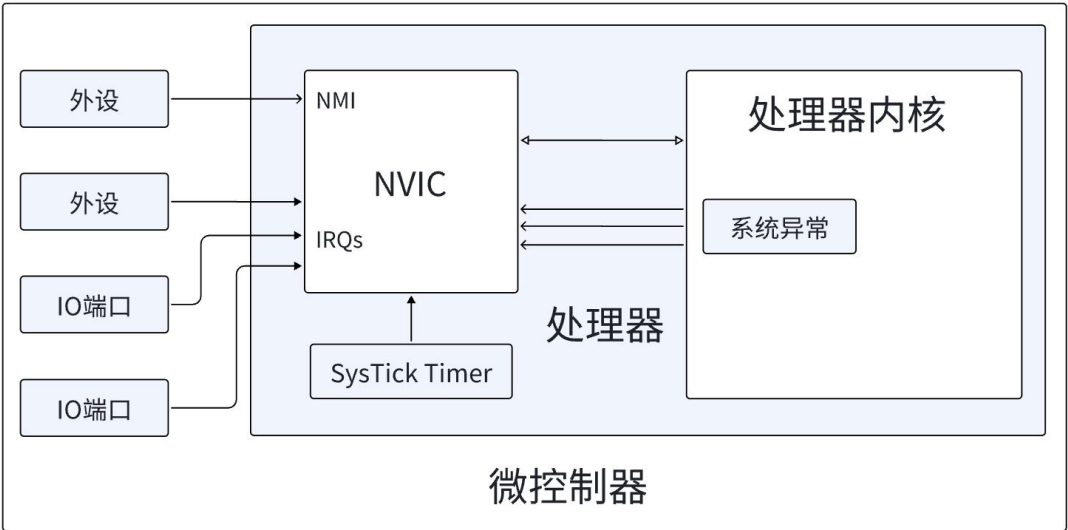


图 2-9 典型 MPU 中的各种异常源

1) NVIC 相关内容

NVIC 在软件的角度上看，是一个被映射成外设的处理器内部模块，编程人员可以像访问外设一样去操控配置 NVIC 模块。其具备灵活的中断和异常管理功能，且通过可编程优先级支持中断嵌套（抢占）。

表 2-1 用于中断控制的 NVIC 寄存器列表

地址	寄存器	CMSIS-Core 符号	功能
0xE000E100~ 0xE000E11C	中断设置使能寄存器	NVIC->ISER[0] ~ NVIC->ISER[7]	写 1 设置中断使能位

0xE000E180~ 0xE000E19C	中断清除使能寄存器	NVIC->ICER[0] ~ NVIC->ICER[7]	写 1 清除中断使能位
0xE000E200~ 0xE000E21C	中断设置挂起寄存器	NVIC->ISPR[0] ~ NVIC->ISPR[7]	写 1 设置中断挂起状态位
0xE000E280~ 0xE000E29C	中断清除挂起寄存器	NVIC->ICPR[0] ~ NVIC->ICPR[7]	写 1 清除中断挂起状态位
0xE000E300~ 0xE000E31C	中断活跃位寄存器	NVIC->IABR[0] ~ NVIC->IABR[7]	指示中断当前活跃状态, 只读
0xE000E400~ 0xE000E4EF	中断优先级寄存器	NVIC->IP[0] ~ NVIC->IP[239]	配置每个中断的优先级 (8 位宽)
0xE000EF00	软件触发中断寄存器	NVIC->STIR	写入中断编号(0-239)以设置相应中断的挂起状态

2) EXTI 外设

EXIT 外设为 Cortex-M4 内核外的一个功能模块, 负责响应所有外部中断信号。在下图中, 中断信号来自于输入线, 通过边沿检测电路捕捉到后, 如果中断屏蔽寄存器未屏蔽该中断, 则信号输入中断挂起请求寄存器, 然后传至 NVIC 中进行处理。

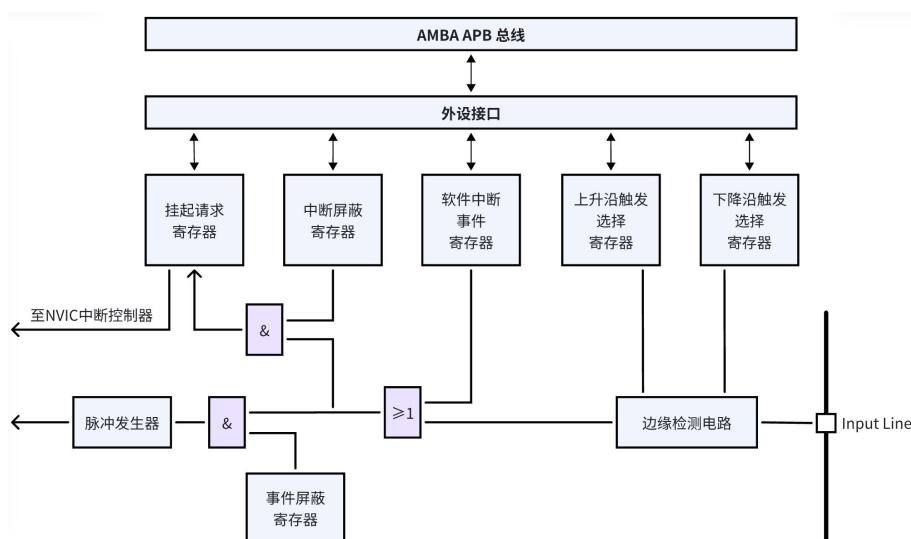


图 2-10 外部中断/事件控制器框图

EXIT 通过外部中断/事件线映射的方式，将 140 个 GPIO 口通过分组式方法链接到 EXIT 线上，用于收集中断信号。

3) 中断处理流程

中断输入和挂起：挂起即在处理中断 A 的时候，把中断 A 禁止了，然后在退出中断 A 的处理之前，又来了一个 A，则 A 自动被挂起，中断处于挂起状态，“等待处理”、“排队”，不会一直产生请求信号，或者处理中途请求撤销这种情况。中断入口处，多个寄存器会被压栈，同时，ISR 的起始地址会被从向量表中取出。处理完后，执行“异常返回”，之前压栈的寄存器纷纷出栈，被中断程序继续执行。中断活跃状态被清除。

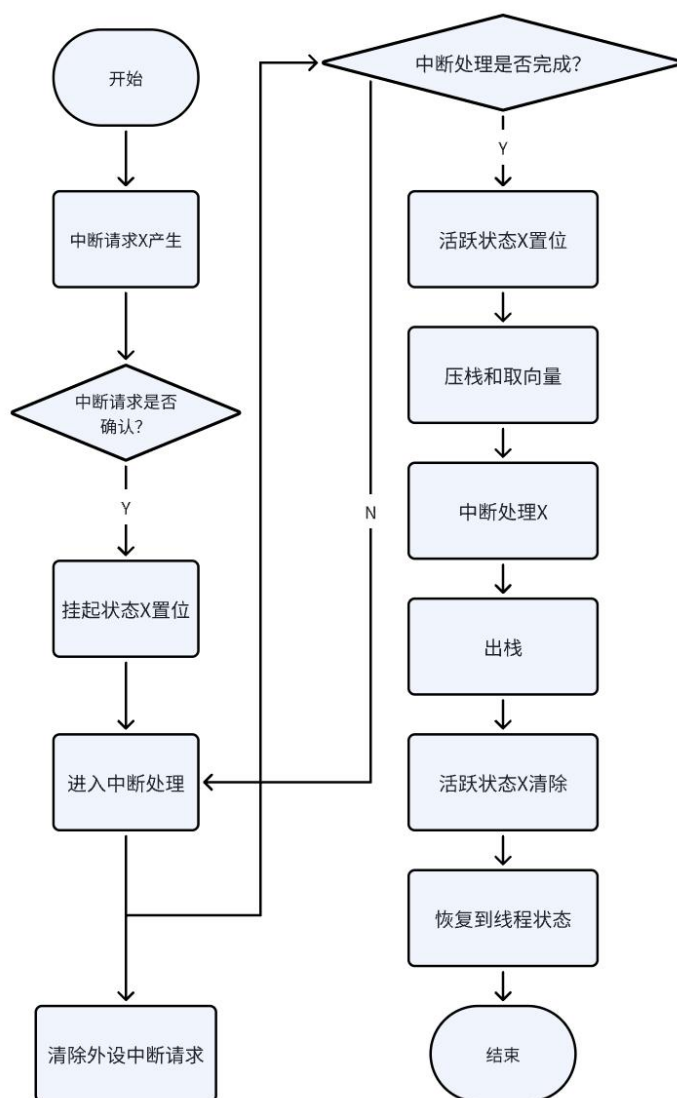


图 2-11 中断挂起和激活行为

2.2.2 任务堆栈初始化

堆栈是一种用来保存任务现场的数据结构，任务在运行时必须有一个完整的堆栈。如果堆栈没有被正确地初始化，会导致任务运行时出现各种异常。因此，在任务初始化时，需要对任务的堆栈进行正确的初始化。

堆栈在内存中被模拟成一块区域，主要用于保存 CPU 的所有寄存器和任务的入口函数地址。当操作系统要建立一个任务时，首先需要对任务的堆栈空间进行初始化。在 $\mu\text{C}/\text{OS-II}$ 中，任务的堆栈初始化主要通过 `OSTaskStkInit()` 函数实现，其函数的原型为：`OS_STK *OSTaskStkInit(void (task)(voidp_arg), void *p_arg, OS_STK *ptos, INT16U opt)`。其中，`task` 参数是任务执行函数的入口地址，`p_arg` 是任务参数的地址，`ptos` 是堆栈顶部指针，`opt` 是预留参数，函数最终返回初始化后的堆栈指针。

在堆栈初始化过程中，需要保存任务的起始地址、处理器状态字、`EXC_RETURN` 参数和一些寄存器等信息。根据 STM32 的寄存器结构，堆栈初始化函数需要按照特定的顺序保存寄存器的值，以保证运行环境能够正确恢复。因此，在进行上下文切换时，需要严格按照模拟压栈的顺序来保存和恢复寄存器的值。

代码 2-7 OSTaskStkInit 函数

```
OS_STK *OSTaskStkInit(void (*task)(void *p_arg),
                      void *p_arg, OS_STK *ptos,
                      INT16U opt) {
    OS_STK *stk;
    opt = opt;
    stk = (OS_STK *)((OS_STK)(ptos) & 0xFFFFFFF8u); // 双字对齐
    *--stk = (OS_STK)0x01000000uL; // xPSR 寄存器
    *--stk = (OS_STK)task; // PC 寄存器，指向任务函数的地址
    *--stk = (OS_STK)0xFFFFFFFFL; // LR 寄存器，设置为异常返回地址
    *--stk = (OS_STK)0x12; // R12 寄存器
    *--stk = (OS_STK)0x3; // R3 寄存器
    *--stk = (OS_STK)0x2; // R2 寄存器
    *--stk = (OS_STK)0x1; // R1 寄存器
    *--stk = (OS_STK)p_arg; // R0 寄存器，任务函数的参数
    *--stk = (OS_STK)0xFFFFFFFFD; // EXEC_RETURN 寄存器，异常返回值
    *--stk = (OS_STK)0x11; // R11 寄存器
    *--stk = (OS_STK)0x10; // R10 寄存器
    *--stk = (OS_STK)0x9; // R9 寄存器
```

```

*(--stk) = (OS_STK)0x8;           // R8 寄存器
*(--stk) = (OS_STK)0x7;           // R7 寄存器
*(--stk) = (OS_STK)0x6;           // R6 寄存器
*(--stk) = (OS_STK)0x5;           // R5 寄存器
*(--stk) = (OS_STK)0x4;           // R4 寄存器
return stk;                         // 返回栈指针
}

```

如代码 1-12 所示，在 OSTaskStkInit 函数中我们首先通过对任务堆栈指针进行位操作，确保堆栈地址按照 AAPCS 规则进行了双字对齐，这样可以保证在使用堆栈时不会发生错误。在 STM32 任务切换时，硬件会将 xPSR、PC、LR、R12、R0-R3 寄存器自动入栈，即按照硬件寄存器入栈顺序进行寄存器保存。后续将 R11-R4 通用寄存器顺序入栈，最终返回最新的 stk 指针。

2.3 ARM 内核时间驱动

2.3.1 时间驱动原理

根据 1.2.3 小节中的内容，我们已知中断是用于处理预设的突发事件，但上面讨论的事件是无规律的，只知道中断可能发生，而不知道何时发生。因此我们需要在此基础上引入计时器。其使得我们可以手动配置其频率（或者说时间间隔），以达到计时的功能。每过一段时间，就发起一次中断，进行对应的中断处理。基于此，我们可以周期性地执行一些特定的函数，便可以实现类似计时器的功能。

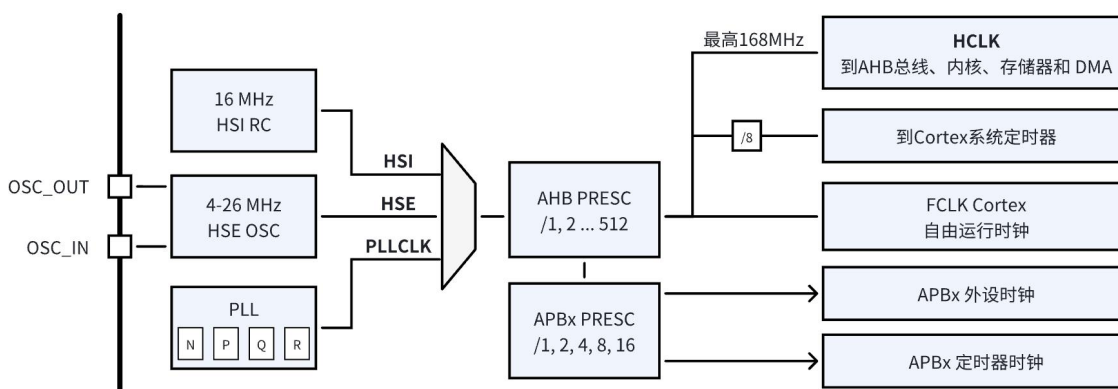


图 2-12 时钟树示意图

在具体实现中，我们首先要了解 STM32 的时钟频率是如何得到的。如图 1-11 所示，我们最后输出的总线频率 SYSCLK 依赖前面 SW 所选择的频率。SW 的选择有三种，HSI 为 RC 振荡器，精度不高，但上电自动启动，可用于执行最开

始的代码，调节频率；HSE 为外部晶振，精度较高；PLLCLK 为锁相环时钟，可以用于调频，。在芯片开始启动时，先使用 HSI 的时钟，执行较慢。打开 HSE，并配好 PLL 的 P.Q.R.N 系数后，等待 PLL 输出到 SW，此时通过操作寄存器配置为选择输出便可以将此作为芯片运行时钟以及总线上的时钟了。

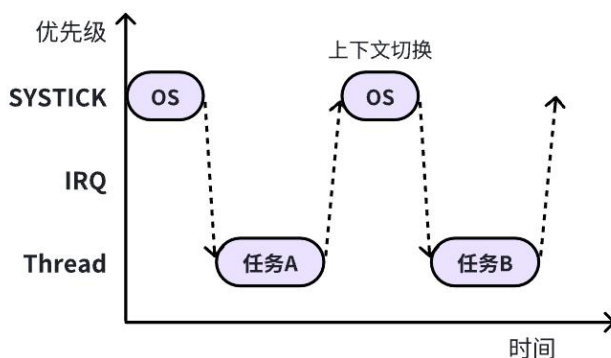


图 2-13 任务间通过 SysTick 轮转调度的简单模式

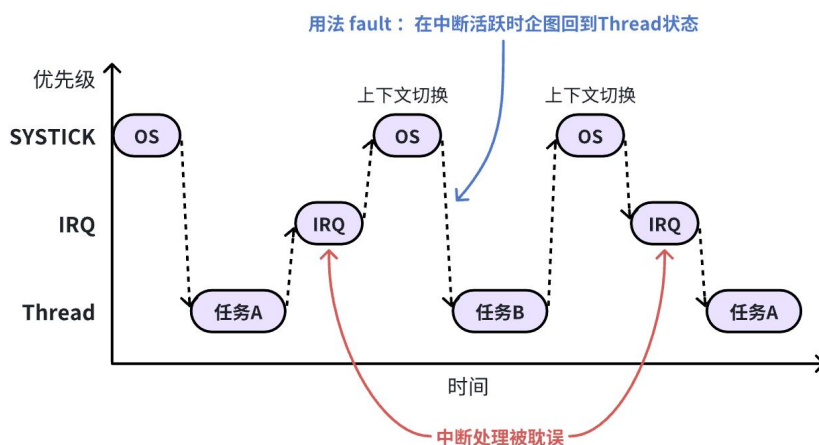


图 2-14 发生 IRQ 时上下文切换的问题

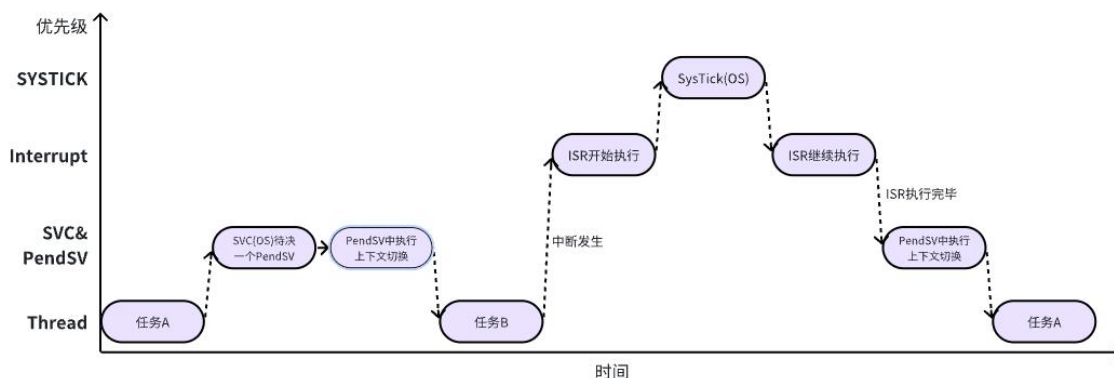


图 2-15 使用 PendSV 控制上下文切换

接下来需配置 STM32 单片机的心跳 ——Systick，其时钟源于 AHB，频率与设定的 SW 输出频率一致。接着我们需要对 SK_CTRL 寄存器进行配置，选择不分频，设置计数器到 0 时产生中断请求，同时使能 Systick。此外我们还需配置 STK_LOAD 寄存器，用于储存时基个数。Systick 的周期等于时基个数乘以一个时基的时长（频率的倒数），根据计算得出的个数存入该寄存器。有了中断信号后，要编写中断服务函数，但此中断服务函数大部分不在 Systick 中断中执行，而是在 pendSV 中，如图 1-13 所示，Systick 是最高优先级任务，不能长时间停留，否则会耽误其他中断任务执行。所以在 Systick 中完成必要操作后，要及时转到优先级较低的 pendSV 中，这样其他中断到来时才能做出响应，如图 1-14 所示，需要部分执行代码放在低优先级中，以便其他中断打断。

2.3.2 时间驱动实现

2.3.2.1 时钟配置

我们组选择在 os_cpu.s 文件里实现系统时钟初始化，主要原因是，保证在系统启动时，会在 os_cpu.s 中执行系统调用函数，将 system_Init 函数配置在其中，可以在启动阶段的部分完成时钟初始化，保证各任务顺利运行。

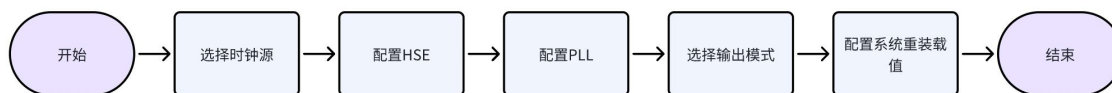


图 2-16 系统时钟配置流程图

代码 2-8 时钟初始化

```

SystemInit
    LDR R0, =RCC_CR //选择时钟源
    LDR R1, [R0]
    ORR R1, R1, #(1 << 16) :or: (1 << 18)
    STR R1, [R0]

Wait_HSE_Ready
    LDR R1, [R0] //选择 HSE 作为系统时钟源
    TST R1, #(1 << 17)
    BEQ Wait_HSE_Ready

    LDR R0, =FLASH_ACR //配置 ACR 寄存器
    MOV R1, #0x02
    STR R1, [R0]
  
```

```

LDR R0, =RCC_PLLCFGR //配置锁相环 PLL
MOV R1, #0x00000000
ORR R1, R1, #(8 << 0) ; PLLM = 16 (bits 5:0)
ORR R1, R1, #(336 << 6) ; PLLN = 336 (bits 14:6)
ORR R1, R1, #(1 << 16) ; PLLP = 0b00
ORR R1, R1, #(1 << 22) ; PLLSRC = 1
STR R1, [R0]

LDR R0, =RCC_CR
LDR R1, [R0]
ORR R1, R1, #(1 << 24) //配置系统时钟模式为锁相环
STR R1, [R0]

Wait_PLL_Ready
LDR R1, [R0]
TST R1, #(1 << 25)
BEQ Wait_PLL_Ready

LDR R0, =RCC_CFGR
MOV R1, #0x00000000
ORR R1, R1, #(0 << 4) ; HPRE=0
ORR R1, R1, #(0x4 << 10) ; PPRE1=0b100
ORR R1, R1, #(0 << 13) ; PPRE2=0
ORR R1, R1, #(2 << 0) ; SW=0b10
STR R1, [R0]

Wait_Clock_Switch
LDR R1, [R0]
AND R2, R1, #0xC
CMP R2, #0x8
BNE Wait_Clock_Switch

```

在完成初始化后我们正式进入配置部分，我们主要针对系统时钟的时基单元包括 ARR、CCR 等值进行配置

代码 2-9 时基单元部分

```

LDR R0, =0xE000E010
MOV R1, #0x00000000
ORR R1, #5

```

```

STR R1, [R0]
LDR R0, =SYSTICK_BASE
LDR R1, =83999
STR R1, [R0, #0x04]
MOV R1, #0
STR R1, [R0, #0x08]
MOV R1, #0x07
STR R1, [R0, #0x00]
LDR R0, =SCB_SHPR3
    ORR R1, #(0x5 << 24)
    STR R1, [R0]

```

2.3.2.2 SysTick 中断配置

Systick 定时器就是系统滴答定时器，一个 24 位的倒计时（从大到小）定时器，计到 0 时，将从 RELOAD 寄存器中自动重装载定时初值。移植 $\mu\text{C}/\text{OS-II}$ 需要使用 Systick 来作为系统时钟。配置 SYSTICK 主要通过 My_Systick_Config() 和 SysTick_Handler() 两个函数来完成。

代码 2-10 My_Systick_Config() 函数实现

```

void My_Systick_Config(uint32_t reload_value){
    SysTick->LOAD |= reload_value-1;
    SysTick->VAL  |= SysTick->LOAD;
    SysTick->CTRL |= 1<<2;
    SysTick->CTRL |= 1<<1;
    SysTick->CTRL |= 0x01;
}

```

代码 2-11 SysTick_Handler() 函数实现

```

SysTick_Handler
    ; OS_SaveSR
    cpsid i
    mrs r0, BASEPRI          ; save previous BASEPRI
    mov r1, #0               ; set new BASEPRI
    msr BASEPRI, r1
    dsb
    isb
    cpsie i

    ; OSIntEnter
    push {lr}
    bl OSIntEnter

```

```
pop {lr}

; OS_RestoreSR
cpsid i
msr BASEPRI, r0
dsb
isb
cpsie i

; if OSIntNesting == 1
ldr r1, =OSIntNesting
ldrb r2, [r1]          ; OSIntNesting : int8u
cmp r2, #1
beq OSTickISR_FromTask
bne OSTickISR_FromInt

OSTickISR_FromTask
; save remained registers & save psp
mrs r0, psp
stmfd r0!, {r4-r11, lr}
msr psp, r0
ldr r2, =OSTCBCur
ldr r3, [r2]
str r0, [r3]
bl OSTimeTick
bl OSIntExit

; restore remained registers
mrs r0, psp
ldmfd r0!, {r4-r11, lr}
msr psp, r0
bx lr

OSTickISR_FromInt
; save remained registers
mrs r0, msp
stmfd r0!, {r4-r11, lr}
msr msp, r0
bl OSTimeTick
bl OSIntExit
```



```

; restore remained registers
mrs r0, msp
ldmfd r0!, {r4-r11, lr}
msr msp, r0
bx lr

```

My_Systick_Config 函数的作用就是重载值初始化和配置 SysTick 定时器，将时钟源设置为 AHB，启用中断，并启动计数器。SysTick_Handler 函数的作用是保存和恢复 BASEPRI 寄存器，调用 OSIntEnter 函数，以及检查 OSIntNesting 的值以确定执行路径，如果 OSIntNesting 等于 1，意味着 SysTick 中断发生时，系统已经在为一个中断服务。在这种情况下，程序会调用 STickISR_FromTask，在其中会进行额外的保存原始任务的堆栈指针的 PSP 工作；否则它将调用 OSTickISR_FromInt，确保整个函数在 MSP 的操纵下执行。

2.4 μ C/OS II 启动代码设计

实际开发过程中的启动分为两部分，第一部分是开发板的启动，通过中断向量表的 reset 中断实现；另一部分是 OS 的初始化和启动。OS 的初始化代码已经实现，我们只需重点关注开发板硬件的启动流程和配置，具体流程图如下图所示：



图 2-17 启动流程设计

μ C/OS II 系统有多个堆栈，包括一个系统堆栈和多个线程堆栈。故而在整个代码运行前需要先初始化这个系统堆栈，这样后面的系统才能够正常运行。在此处，我们需要手动的给芯片分配栈空间，并调整 SP 指针，供后续代码使用。

如在代码 1-3 L(1)代码表示 Stack_Size 大小为 0x400, L(2)表示此段存储空间为栈,可读可写,按 3 字节对齐。在 L(3)中,真正分配实际具体空间给标号 Stack_Mem。L(4)和 L(5)的作用同理,分配堆空间 0x200,可读可写,按 3 字节对齐,分配给标号 Heap_Mem 的变量。

代码 2-12 系统堆栈初始化

Stack_Size	EQU	0x00000400	(1)
	AREA	STACK, NOINIT, READWRITE, ALIGN=3	(2)
Stack_Mem	SPACE	Stack_Size	(3)

```

__initial_sp
Heap_Size      EQU      0x00000200                (4)
               AREA     HEAP, NOINIT, READWRITE, ALIGN=3
__heap_base
Heap_Mem SPACE Heap_Size                (5)

```

重新上电时，由硬件控制，PC 会自动跳转到 Reset_Handler 执行复位中断函数，这里包含了 SystemInit 时钟频率配置和我们自行定义的分散加载文件 myScatterLoad。

代码 2-13 Reset_Handler 复位中断

```

Reset_Handler  PROC
                 EXPORT Reset_Handler             [WEAK]
                 IMPORT SystemInit
                 IMPORT myScatterLoad
                 IMPORT main

                 LDR    R0, =SystemInit
                 BLX    R0

                 LDR    R0, =myScatterLoad
                 BLX    R0

                 BL     main
                 B       .
                 ENDP

```

在时钟频率配置部分，为了能增加计算能力，我们需要更改对应频率，也就是 1.2.4 章节中的修改 SYSCCLK 的时钟源。为此，我们小组在 os_cpu_a.asm 文件中编写了 SystemInit 函数，用于修改时钟频率，在这个函数中，我们要实现的便是调配 PLL 锁相环时钟频率以及系统时基单元的初始化。

代码 2-14 SystemInit 时钟频率配置

```

SystemInit
    LDR R0, =RCC_CR
    LDR R1, [R0]                ; 读取当前 RCC_CR 的值
    ORR R1, R1, #(1 << 16) :or: (1 << 18)
    ; 若使用外部时钟源，添加: ORR R1, R1, #(1 << 18)
    STR R1, [R0]

```

```
Wait_HSE_Ready
    LDR R1, [R0]
    TST R1, #(1 << 17)
    BEQ Wait_HSE_Ready

;-----
; 2.
;-----

    LDR R0, =FLASH_ACR
    MOV R1, #0x02
    STR R1, [R0]

;-----
; 3.
;-----

    LDR R0, =RCC_PLLCFGR
    MOV R1, #0x00000000
    ORR R1, R1, #(8 << 0)      ; PLLM = 16 (bits 5:0)
    ORR R1, R1, #(336 << 6)    ; PLLN = 336 (bits 14:6)
    ORR R1, R1, #(1 << 16)     ; PLLP = 0b00
    ORR R1, R1, #(1 << 22)     ; PLLSRC = 1
    STR R1, [R0]

;-----
; 4.
;-----

    LDR R0, =RCC_CR
    LDR R1, [R0]
    ORR R1, R1, #(1 << 24)
    STR R1, [R0]

Wait_PLL_Ready
    LDR R1, [R0]
    TST R1, #(1 << 25)
    BEQ Wait_PLL_Ready

;-----
; 5.
;-----

    LDR R0, =RCC_CFGR
    MOV R1, #0x00000000
```

```

    ORR R1, R1, #(0 << 4)      ; HPRE=0
    ORR R1, R1, #(0x4 << 10)   ; PPRE1=0b100
    ORR R1, R1, #(0 << 13)    ; PPRE2=0
    ORR R1, R1, #(2 << 0)     ; SW=0b10
    STR R1, [R0]

Wait_Clock_Switch
    LDR R1, [R0]
    AND R2, R1, #0xC
    CMP R2, #0x8
    BNE Wait_Clock_Switch

; -----
; 6.
; -----

    LDR R0, =0xE000E010
    MOV R1, #0x00000000
    ORR R1, #5
    STR R1, [R0]

    LDR R0, =SYSTICK_BASE
    LDR R1, =83999
    STR R1, [R0, #0x04]
    MOV R1, #0
    STR R1, [R0, #0x08]
    MOV R1, #0x07
    STR R1, [R0, #0x00]

    LDR R0, =SCB_SHPR3
    ORR R1, #(0x5 << 24)
    STR R1, [R0]
    CPSIE I
    BX LR

ALIGN
END

```

时钟初始化过后就是代码拷贝和清除，在此部分我们设计代码拷贝为 4 字节对齐的拷贝规则，实现 DATA 段复制，而清除 BSS 段则选择自上而下 4 字节对齐的清除方式。

代码 2-15 myScatterload 分散加载和清除

```
AREA |.text|,CODE,READONLY
myScatterLoad PROC
    EXPORT myScatterLoad
    IMPORT |Image$$RW_IRAM1$$Base|
    IMPORT |Image$$RW_IRAM1$$Length|
    IMPORT |Load$$RW_IRAM1$$Base|
    IMPORT |Image$$RW_IRAM1$$ZI$$Base|
    IMPORT |Image$$RW_IRAM1$$ZI$$Length|

    ; 复制数据段
    LDR R0, = |Load$$RW_IRAM1$$Base|
    LDR R1, = |Image$$RW_IRAM1$$Base|
    LDR R2, = |Image$$RW_IRAM1$$Length|
CopyData
    SUB R2, R2, #4
    LDR R3, [R0, R2]
    STR R3, [R1, R2]
    CMP R2, #0
    BNE CopyData

    ; 清除 BSS 段
    LDR R0, = |Image$$RW_IRAM1$$ZI$$Base|
    LDR R1, = |Image$$RW_IRAM1$$ZI$$Length|
CleanBss
    SUB R1, R1, #4
    MOV R3, #0
    STR R3, [R0, R1]
    CMP R1, #0
    BNE CleanBss
    ALIGN
    ENDP
END
```

完成以上步骤，系统进入我们编写的主函数，启动 μ C/OS 以及 SytemView 后开始执行相应的多个任务。

代码 2-16 main 函数编写

```
int main(void) {  
    OSInit();  
  
    SEGGER_SYSVIEW_Conf();  
    SEGGER_SYSVIEW_Start();  
  
    OSStart();  
}
```

第三章 前期完成度和后续实施计划

在项目的前期阶段，我们已顺利完成大部分任务，包括重建工程项目、实现事件驱动和时间驱动机制、完成启动代码编写等。而针对当前存在的问题，我们将在后续阶段集中精力进行优化。

后续，我们将在解决前期问题的基础上，继续实现任务上下文切换机制，并着手完整移植 $\mu\text{C}/\text{OS-II}$ ，包括无人机任务规划和多任务调度设计等。最后，我们将利用 SystemView 完成对系统的测试工作，确保任务切换轮转的正常执行；最后在完成系统测试后，将操作系统置于真实的无人机平台测试，确保其能够按照设计要求正常工作。

参考文献

- [1] 廖勇, 杨霞. 嵌入式操作系统[M]. 人民邮电出版社. 2017.
- [2] 汤小丹, 梁红兵, 哲凤屏等. 操作系统. 西安电子科技大学出版社[M]. 2018.
- [3] Joseph Yiu. The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors[M]. Elsevier Inc. 2014.
- [4] Jean J. Labrosse. μ C/OS-II User Manual. Micrium Press. 2015.
- [5] SEGGER. SystemView User Guide[M]. SEGGER Microcontroller GmbH. 2023.
- [6] STM32F4xx 中文参考手册[J]. 意法半导体
- [7] STM32 固件库使用手册[J]. 意法半导体
- [8] STM32F4 开发指南-寄存器版本[J]. 正点原子
- [9] 零死角玩转 STM32F401[J].